Split-Sync App

Shared Expense Management System

Research Document

Started 02/10/2025

Student Name: Emmanuel Abolade

Student Number: C00288657

Institution: South East technological University, Carlow

Programme: BSc Software Development

Supervisor: Joseph Kehoe

Submitted: 28/11/2025

Table of Content

Table of Content....2
Abstract....4
1.0 Introduction....4
1.1 Reasons for Project....5
1.2 Aims and Objectives of Project....5
1.3 Scope of Project....5
2.0 Review of Existing Applications....6
2.1 Tikkie....6
2.2 Settle up....7
2.3 Revolut....8
2.4 Wise....9
2.5 Splitwise....10
3.0 Technology Stack and Options....12
3.1 Core Frontend Framework Options....12
3.1.1 React.js / React Native....13
3.1.2 Vue.js / NativeScript-Vue / Vue Native....13
3.1.3 Angular / NativeScript / Ionic....14
3.1.4 Svelte / Svelte Native....15
3.1.5 Flutter....15
3.1.7 Frontend Framework Comparison....17
3.1.8 Progressive Web App (PWA)....19
3.1.9 Core Frontend Framework Choice Summary....19
3.2 UI/Styling Frameworks Options....20
3.2.1 Tailwind CSS....20
3.2.2 Material UI....20
3.2.3 Bootstrap....21
3.2.4 Comparison of UI/Styling Framework Options....21
3.2.5 UI/Styling Framework Choice Summary....22
3.3 Backend Framework Options....23
3.3.1 Node.js....23
3.3.2 Django....23
3.3.3 Ruby on Rails....24
3.3.4 Spring Boot....24
3.3.5 ASP.NET Core....24

3.4 Comparison of Backend Framework Options....24
3.5 Framework Choice Summary....26
4.0 Database Options....27
4.1 Selection criteria (what SplitSync needs)....27
5.0 Payment Gateway Integration....29
5.1 How different payment gateway differ....29
5.2 How payment gateways work in mobile apps....30
5.3 Popular payment gateway provider options....31
6.0 Security and Safety for Fintech Applications....34
6.1 Authentication and identity management....34
6.2 Data encryption and protection....35
6.3 PCI DSS compliance requirements....36
6.4 API security and protection mechanisms....37
6.5 Real-world implementations and Firebase security....37
7.0 Hosting and Deployment Infrastructure....38
7.1 Platform comparison and selection criteria....38
7.2 CI/CD pipeline implementation....39
7.3 Performance and cost considerations....40
7.4 Deployment architecture and environment management....40
8.0 OCR Receipt Scanning Technologies....41
8.1 Technology landscape and accuracy comparison....41
8.2 Implementation complexity and integration patterns....42
8.3 Cost analysis and recommendations....43
8.4 Technology recommendations for Split-Sync....43
9.0 State Management Solutions for React Applications....44
9.1 Context API: React's native solution....44
9.2 Redux Toolkit: Enterprise-grade state management....45
9.3 Zustand: Lightweight modern alternative....46
9.4 Comparative analysis and recommendations....47
10.0 Offline Support Strategies and Implementation....48
10.1 Progressive Web App architecture....48
10.2 Service worker caching strategies....49
10.3 Data synchronization and conflict resolution....49
10.4 Firebase offline capabilities and limitations....50
10.5 Security implications of offline storage....51
10.6 User experience patterns for offline states....52
11.0 Essential Development Tools and Workflow....53
11.1 Version control with Git and branching strategies....53

11.2 Testing strategy and framework selection....53
11.3 API documentation with Swagger and OpenAPI.... 54
11.4 Monitoring, error tracking, and observability....55
11.5 Analytics implementation with privacy consideration....55
11.6 Development workflow automation....56
11.7 Dependency management and security....57
12. Conclusion.... 58
Appendix.... 59
References.... 59

Abstract

Consumers are moving from cash to cards, mobile wallets, and bank-to-bank digital payments, with mobile devices now the default touchpoint. Yet in everyday life managing funds in small groups such as roommates, couples, event planners, travel teams etc. remains awkward and prone to mistakes. This document examines prominent shared-finance applications - Tikkie, Settle Up, Revolut, Wise, Splitwise, etc., analyzes the regulatory and adoption patterns driving cashless, mobile-first payments in Europe and beyond, and presents a dedicated solution: SplitSync is a mobile-friendly web application designed to simplify shared expense management among small groups such as roommates, couples, and travel teams like capturing expenses, splitting it fairly, showing who owes whom, and making it easy to settle—without trying to be a bank or an investing platform. The app will enable users to track group transactions in real time, automate expense input via OCR receipt scanning, and minimize settlement complexity using smart algorithms. This research document explores the technologies, platforms, and design decisions behind SplitSync, including frontend and backend frameworks, database options, real-time communication protocols, and payment integration. It also aims to learn from existing apps and make pragmatic technology choices that deliver a polished minimum viable product without heavy operational burden. The goal is to demonstrate a clear understanding of the technical landscape and justify the choices made for implementation.

1.0 Introduction

In recent years, the popularity of shared digital finance tools such as Splitwise, Tikkie, and Settle up has surged, reflecting a broader shift toward cashless payments and mobile-first financial interactions. The use of ATMs to withdraw for payments has been on decline, down 11% in Q2 2025 compared to the same period last year (BOI, 2005). Koopa H. A. (2024) said that people in the Netherlands mostly use Tikkie to share the costs of dinners, groceries and lunches, which are the three most common descriptions given for Tikkies. As people increasingly rely on digital wallets and contactless transactions, the need for smarter, more transparent expense-sharing

solutions has become evident (UK Finance, 2025). Whether splitting dinner bills, managing travel costs, or coordinating household expenses, users seek tools that simplify the process and reduce friction in financial coordination.

1.1 Reasons for Project

Despite the availability of digital tools, many users still face challenges in managing shared expenses. Common issues include forgetting who paid for what, miscalculating shared bills, delayed reimbursements, and a lack of transparency in group transactions. Even with good apps out there, people bounce between screenshots, spreadsheets, and awkward reminders. These problems can lead to confusion, strained relationships, and financial inefficiencies, especially in informal settings where accountability is low and manual tracking is error-prone. The problem is cross-bank, cross-border, and social. That's why you see "split the bill" features popping up in mainstream banking and fintech apps. Our goal is to deliver a scalable, secure, and user-friendly platform that solves real-world expense-sharing problems.

1.2 Aims and Objectives of Project

The specific objectives of this application include:

- Enabling senders to create groups, add expenses, and settle balances.
- Enabling Group Members to participate in shared expenses, track shared expenses, split bills fairly, and settle debts easily.
- Enabling real-time tracking of group transactions and balances.
- Integrating OCR (Optical Character Recognition) receipt scanning for automatic expense input.
- Providing visual representations of debts and smart settlement paths to minimize transaction loops and simplify repayments.

1.3 Scope of Project

SplitSync is tailored for small to medium-sized groups such as friends, roommates, couples, event organizers and travel teams. It focuses on informal financial coordination rather than enterprise-level accounting. The app does not include features related to full banking systems, cryptocurrency payments, or investment tracking. Instead, it emphasizes ease of use, accessibility, and social engagement within everyday expense-sharing scenarios.

2.0 Review of Existing Applications

Several applications currently address shared expense management, each with unique strengths and limitations. This section explores existing systems in the expense-sharing domain and evaluates the key technologies selected for the development of SplitSync. The goal is to justify each technical decision and demonstrate a clear understanding of the tools and platforms involved.

2.1 Tikkie

Tikkie is an initiative of ABN AMRO and is a free mobile payment app popular in the Netherlands that allows users to easily request and send money via a payment link via whatsapp or Quick Response code (ABN AMRO, 2024). It's used by individuals to split bills with friends or by businesses to get paid by customers. The recipient pays the request through their own bank using iDEAL without needing the Tikkie app themselves. Tikkie's tech stack is built around a modern, cloud-native architecture that runs primarily on AWS, using services like ECS (Elastic Container Service), Lambda, DynamoDB, SNS (Simple Notification Service), and SQS (Simple Queue Service) to support a microservices and event-driven design. The backend is developed mainly in Java (Spring Boot) and Node.js, while the frontend uses React with TypeScript for its web interfaces (Stage.nl., 2025). Infrastructure and deployment are managed through AWS CloudFormation and AWS CDK (Cloud Development kit), supported by CI/CD (Continuous Integration and Continuous Deployment) pipelines in tools such as Jenkins or GitHub Actions. For data storage, DynamoDB is used for scalable workloads, likely alongside relational databases for transactional data (Stage.nl., 2025) . Tikkie exposes its functionality via REST APIs secured with API keys and tokens, integrating closely with Dutch payment systems like iDEAL (De Best, R., 2024) . Overall, the stack reflects a scalable, secure, and modular fintech platform optimized for rapid iteration and cloud efficiency.

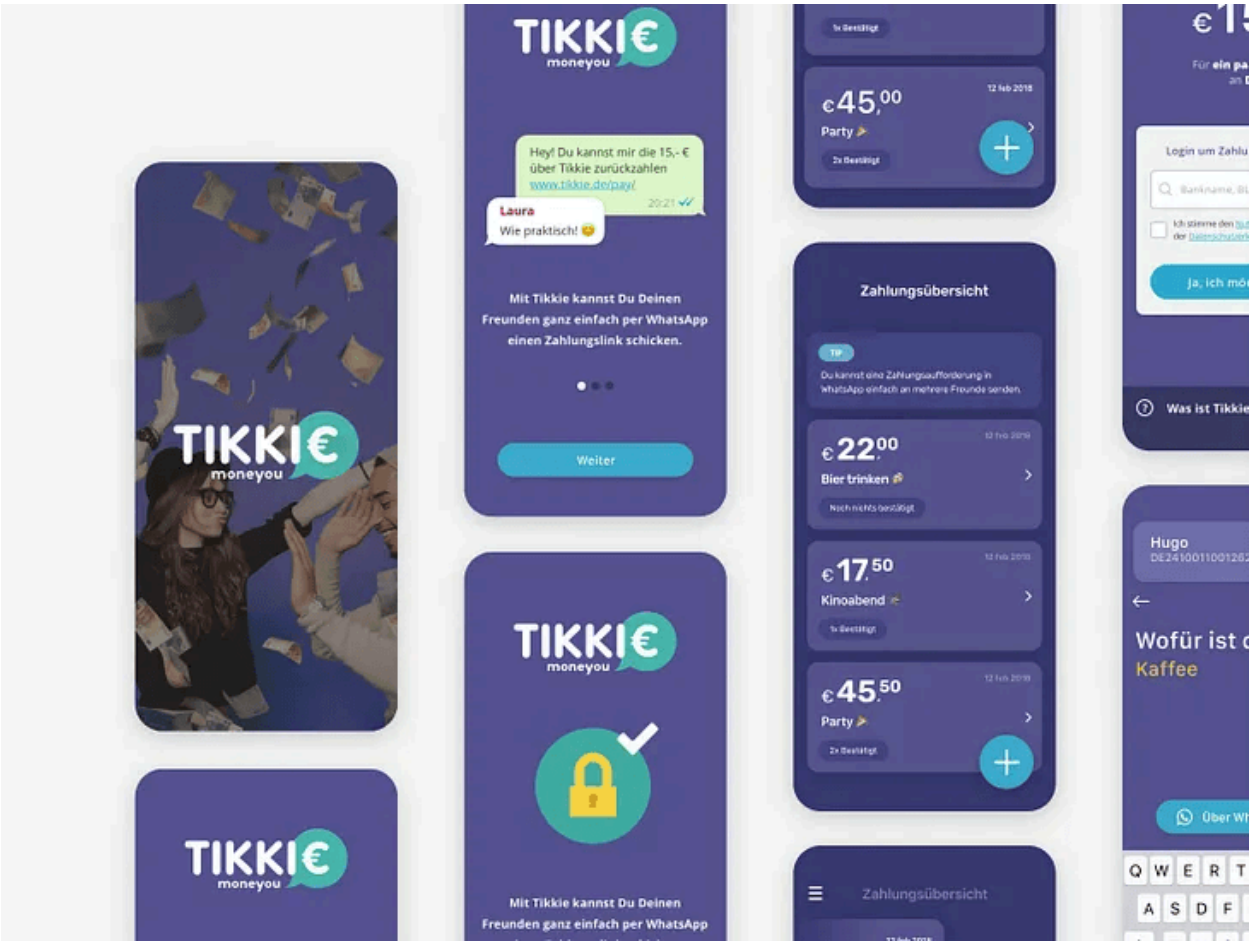


Fig. 1 Tikkie Interfaces (*Source: Hugo, 2025)*

2.2 Settle up

This is a cross-platform group expense tracker with multi-currency, offline mode, recurring expenses, voice-assistant integrations, and minimal-transaction settlement suggestions app (Settle Up., 2025). Settle Up was entirely rewritten at one point to improve synchronization and offline support; its Android version leverages Firebase for real-time data sync and offline caching, enabling updates made on one device to appear almost instantly on others.

The app supports cross-platform use (Android, iOS, web) and works offline, with automatic synchronization when connectivity resumes.

In the redesign, they adopted better account and permission models (Google, Facebook, email/password logins) and synchronized exchange rates for expenses.

While the backend architecture is not publicly disclosed in detail, the reliance on Firebase for sync hints at using Firebase's realtime database or Firestore, plus cloud functions or serverless components for backend logic. The mobile apps are native (Android / iOS) and they follow the

platform conventions (e.g. using Material Design on Android) with native elements (Vávra D., 2017).

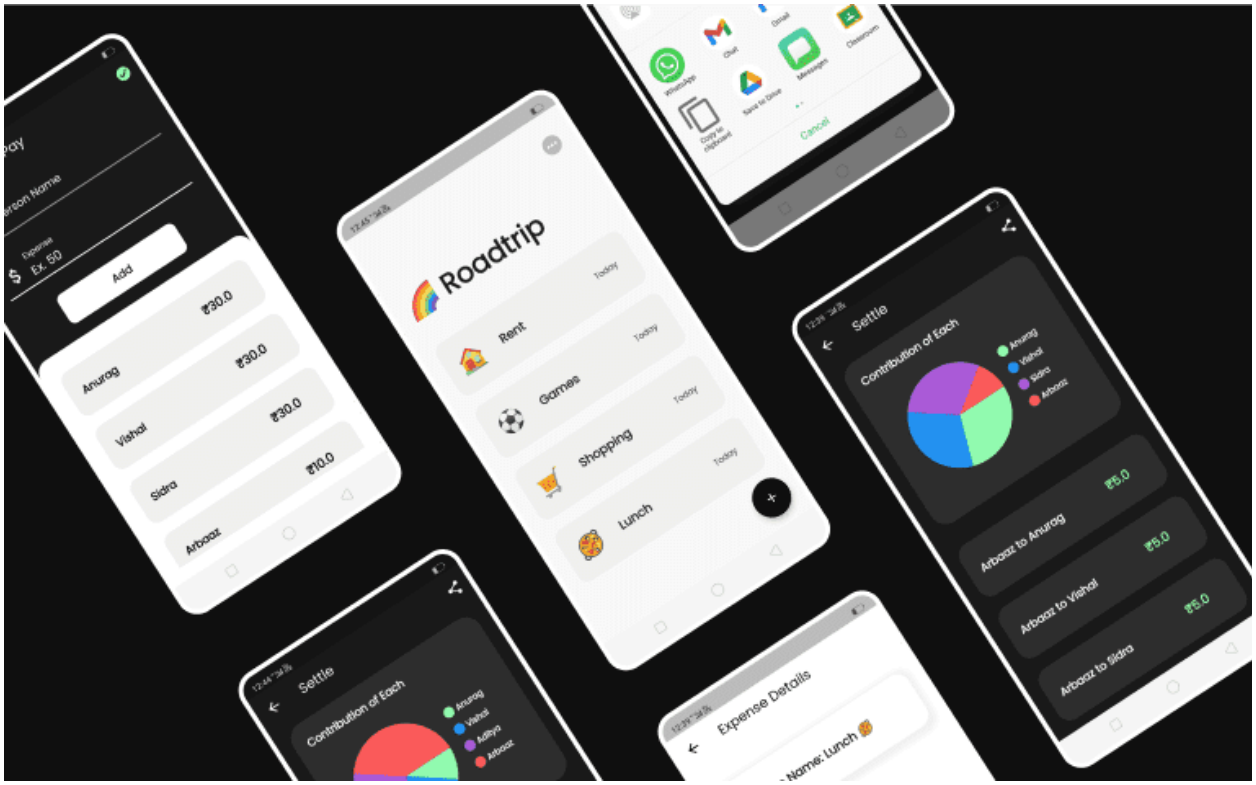


Fig. 2 ScreenShot of Settle Up (*Source: HamzaKhan07 , n.d)*

2.3 Revolut

Revolut is a global financial technology company that offers digital banking, payments, investments, and currency exchange services through its mobile app. Founded in 2015 in the UK, Revolut has grown into one of the largest neobanks in the world, providing users with features like instant money transfers, budgeting tools, crypto trading, and multi-currency accounts. Technically, Revolut operates on a microservices architecture built for scalability and high reliability. Its backend is primarily developed in Java 17 and Kotlin, using frameworks such as SparkJava, jOOQ, and Flyway for database migrations (Revolut, 2025). Data is managed with PostgreSQL (as both the main relational and event store) and Redis for caching. The frontend stack includes ReactJS for web applications, while Google Cloud Platform (GCP) hosts their infrastructure with Docker and Kubernetes (GKE) for container orchestration and deployment (Kairov, 2024). Revolut follows an event-driven design pattern, storing key business events in databases to enable modules like fraud detection, compliance, and transaction analytics to consume events asynchronously (AbnAsia.org, 2024). This combination of technologies enables Revolut to deliver banking-grade reliability while iterating rapidly on new features.

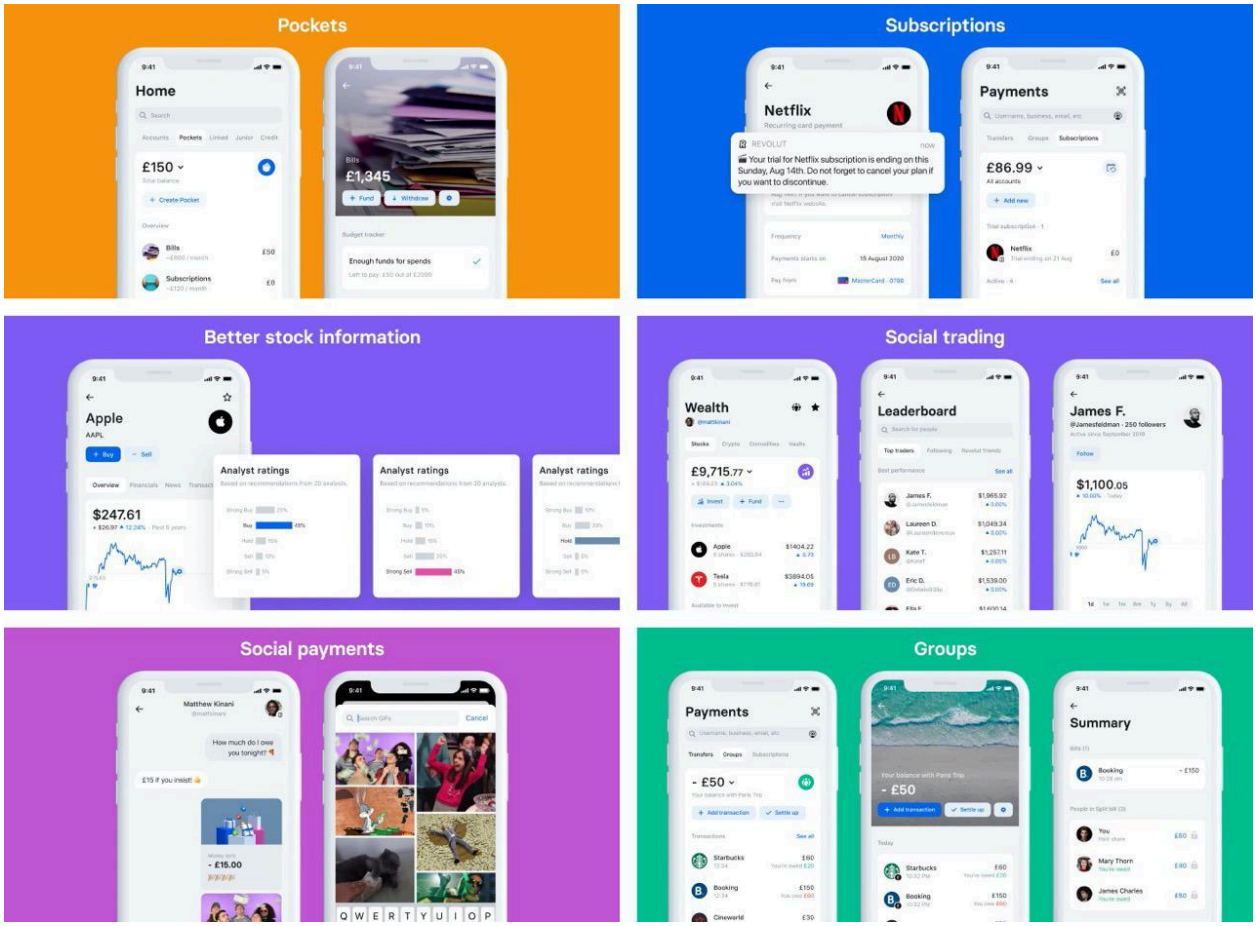


Fig. 3 Revolut Interfaces (Source: Reddit, 2020)

2.4 Wise

Wise (formerly TransferWise) is a global fintech company that facilitates low-cost international money transfers, multi-currency accounts, and border-agnostic financial services. Founded in 2011 and headquartered in London, its mission is "money without borders" – making cross-border transactions fast, transparent, and affordable. On the tech side, Wise runs a large, scalable, cloud-native, microservices architecture. Its stack includes Java and Kotlin for backend services, coordinated via thousands of microservices. They use Kubernetes for orchestration, managed via Terraform, Rancher, Helm, and ArgoCD, with Envoy as a service proxy or part of their service mesh. For mobile apps: on Android they use Kotlin, Jetpack Compose, Coroutines/Flows, and for iOS they've improved build infrastructure (Swift, Swift Package Manager etc.). Their frontend web uses Next.js under a custom abstraction called "CRAB" (handling several discrete apps/components for different Wise product features) with React. For data and messaging, they use Kafka for streaming and inter-service communication, PostgreSQL, MariaDB, Redis for different storage and caching needs. Analytics / ML(Machine Learning) pipelines use Snowflake, data lakes (with Apache Iceberg on S3), Trino, and internal prediction

services. Observability is achieved using tools like Grafana (dashboards/metrics), Loki (logs), Tempo (traces), Mimir, etc. as extracted from James (2025).

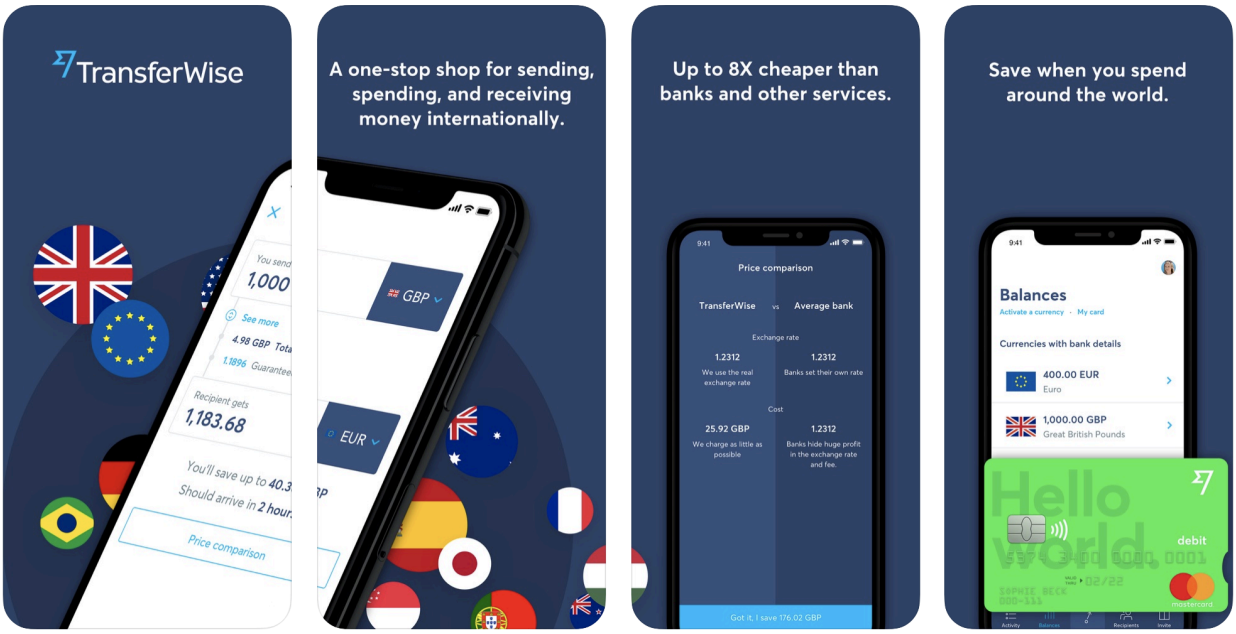


Fig. 4 Wise UI pictures (Source: Huth S., 2024)

2.5 Splitwise

Splitwise is a popular expense-sharing and bill-splitting platform that helps friends, families, and groups track shared expenses and IOUs (I owe You), ensuring everyone pays their fair share. Founded in 2011 and based in Providence, Rhode Island, Splitwise simplifies group finances for activities like travel, rent, and dining, allowing users to log expenses and settle balances through integrations with payment apps such as PayPal and Venmo. Technically, Splitwise's backend is built on Ruby on Rails, which powers its RESTful API and core business logic. The company hosts its infrastructure on AWS and Heroku, providing scalability and easy deployment. The mobile apps are developed natively — Swift (with some legacy Objective-C) for iOS and Kotlin/Java for Android — while the web client is built using modern web technologies like HTML5, CSS3, and JavaScript (Splitwise, 2025). Splitwise follows a continuous deployment model with frequent updates, automated testing, and careful API versioning to maintain backward compatibility. Its stack reflects a clean, reliable, and maintainable design focused on simplicity and usability.

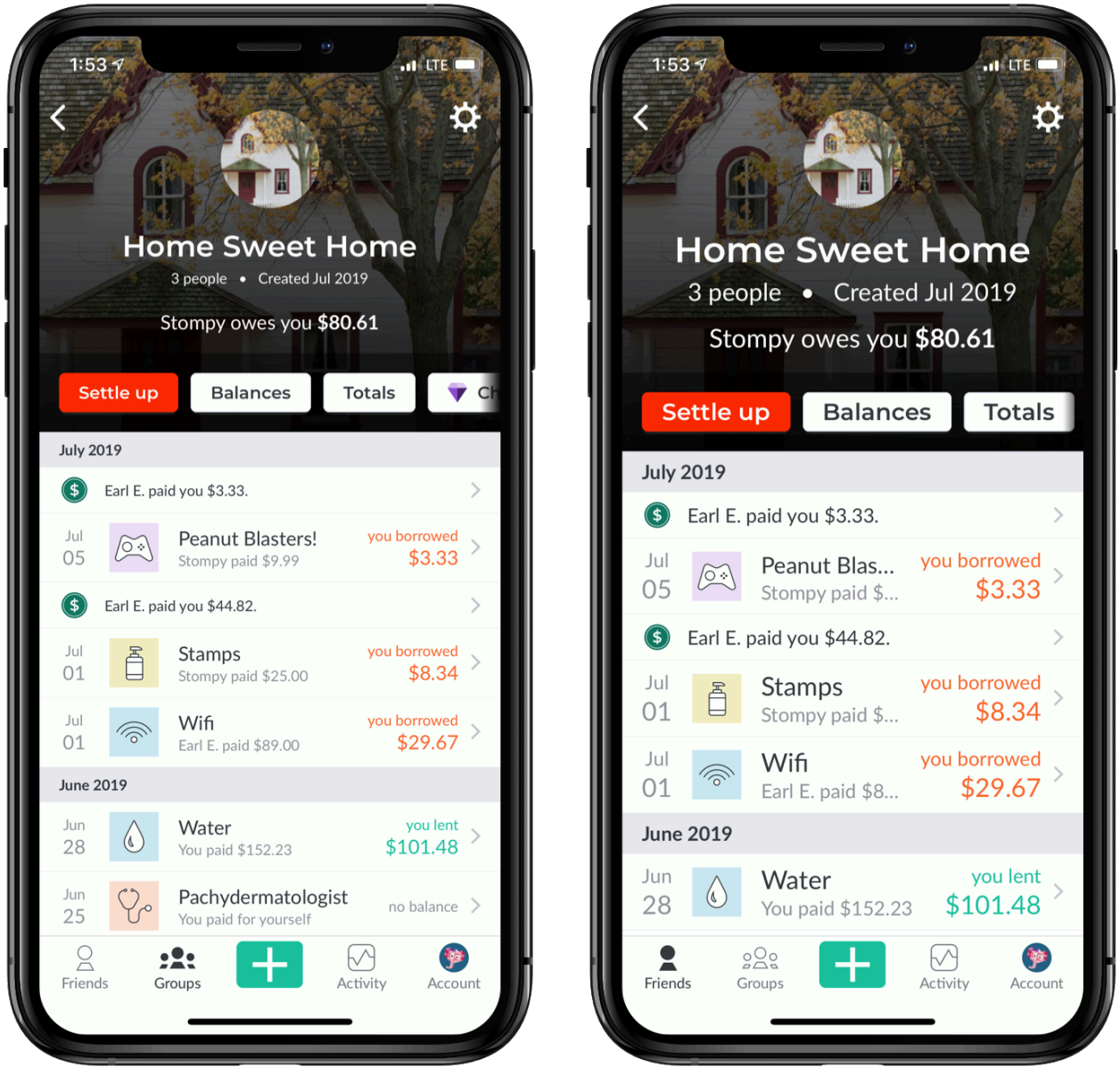


Fig. 5 Splitwise (Source: Jon Bittner, 2025)

The following table compares three prominent tools and highlights insights that inform the design of SplitSync according to Nesbit (2025):

App	Key Features	Weaknesses	Insights for SplitSync
Splitwise	Bill splitting, group tracking	No smart receipt input, no transaction optimization	Adding OCR + graph-based settlement algorithm
Tikkie	Payment links and direct peer-to-peer pay	Limited group management and expense history	Adding shared ledger and visual summary of group balances
Settle Up	Offline mode, multi-currency support	Limited analytics and predictive features	Adding predictive expense summaries

Table 1: Difference between Splitwise, Tikkie and Settle up

Splitwise (2024) remains the most widely used platform for group expense tracking, but its lack of automation and optimization leaves room for improvement. Tikkie (2023) excels in direct payments but lacks collaborative features. Settle Up (2023) offers offline support and currency flexibility, yet its analytical capabilities are minimal. SplitSync aims to combine the best of these systems while introducing smart receipt, better enhancements and real-time collaboration.

3.0 Technology Stack and Options

To develop SplitSync, the goal will be to balance modern, industry-relevant tools with ease of learning, community support, and rapid development. There are several options of tech stack to choose which would be discussed in this chapter.

3.1 Core Frontend Framework Options

Core frameworks for building mobile apps and web apps include: Reactjs/React Native, Vuejs/NativeScript-Vue/Vue Native, Angular/NativeScript/Ionic Framework, Svelte/Svelte Native. Other popular mobile frameworks include Flutter (Using Dart language by google), SwiftUI (iOS)/ Jetpack Compose (Android)

3.1.1 React.js / React Native

React.js, at its core, is a JavaScript library designed for building user interfaces. It's known for its component-based architecture, which means reusable building blocks can be created (components) to make up an entire UI. This approach simplifies development, making it easier to manage and update applications as it grows. With React, dynamic and interactive web applications that respond smoothly to user interactions are created. It's a fantastic choice for modern web development, offering a great developer experience and excellent performance.

React Native on the other hand, is a framework that allows the knowledge of react to build native mobile apps for both iOS and Android. This means code can be written once and deployed on multiple platforms, saving time and effort. React Native utilizes the same component-based approach as React.js, allowing the reuse of code and sharing knowledge between web and mobile projects. It provides a consistent development experience, making it easier to build cross-platform applications with a native look and feel.

Both React.js and React Native thrive on the versatility of JavaScript and TypeScript, providing a strong foundation for both web and mobile applications. They work seamlessly with backend technologies like Node.js and various RESTful APIs, allowing for flexible server-side choices. For data management, they can connect to any database through APIs, with popular options including MongoDB and Firebase. To boost development, supporting libraries like Vite or Next.js for project setup, Axios or Fetch API for backend communication, React Query for data fetching, and Redux Toolkit or the Context API for state management are commonly used.

3.1.2 Vue.js / NativeScript-Vue / Vue Native

Vue.js is a progressive JavaScript framework designed for building user interfaces. It's known for its approachable learning curve, making it a great choice for developers of all skill levels. Vue.js focuses on a clean and concise syntax, which leads to more readable and maintainable code. This framework is perfect for smaller teams or projects where simplicity and ease of use are crucial. Its progressive nature allows features to be adopted as needed, making it flexible for various project sizes and complexities.

Vue.js utilizes JavaScript, TypeScript, and HTML to create user interfaces. It's designed to work seamlessly with various backend frameworks like Node.js and Laravel, providing a smooth integration experience. This flexibility makes it the best backend technology for most projects while still enjoying the benefits of a modern frontend framework. Vue.js applications can connect to any database through APIs, with popular choices including Firebase and MySQL, providing robust data storage and retrieval capabilities (Abramowski, N., 2025).

One of the key strengths of Vue.js is its well-integrated ecosystem. It comes with built-in tools like Vue Router for navigation, Pinia for state management, and Axios for API calls. These tools provide a streamlined development experience, allowing the focus on building application's features rather than dealing with complex setup and configuration. This integrated ecosystem makes Vue.js a lightweight yet powerful option for rapid frontend development without sacrificing performance.

In essence, Vue.js offers a compelling combination of simplicity, readability, and flexibility. Its clean syntax and gentle learning curve make it easy to get started, while its powerful features and integrated ecosystem allow the build of complex applications efficiently. Whether working on a small project or a large-scale application, Vue.js provides a great foundation for modern frontend development.

3.1.3 Angular / NativeScript / Ionic

Angular is a powerful, full-featured framework developed by Google, designed primarily for building large-scale and enterprise-level applications. It offers built-in support for essential features like routing, forms, and TypeScript, which provides a strongly typed environment. This enhances both reliability and maintainability of applications. Angular's opinionated structure promotes consistency across projects, making it particularly beneficial for larger teams where collaboration and code quality are paramount. However, it’s worth noting that Angular has a steeper learning curve compared to frameworks like React or Vue, which may make it less suitable for short-term or small-scale projects, especially for beginners.

NativeScript (Web and Native bridge) is an exciting framework that allows developers to build native mobile applications using Angular (or Vue). Existing web development skills can help create fully functional native apps for iOS and Android. NativeScript empowers developers to access native APIs directly from JavaScript, providing a rich set of functionalities while maintaining the performance and user experience of native applications. This framework is particularly appealing for teams looking to maximize their code reuse across web and mobile platforms, making it a great choice for projects that require a native look and feel without sacrificing development efficiency.

The Ionic Framework (Hybrid App Development) is primarily focused on building hybrid mobile applications. It utilizes web technologies such as HTML, CSS, and JavaScript, and can seamlessly integrate with Angular, React, or Vue. This flexibility allows developers to create high-quality cross-platform applications from a single codebase, significantly reducing development time and effort. Ionic provides a rich library of pre-built UI components, making it easier to design beautiful and responsive applications. Additionally, it supports Progressive Web Apps (PWAs), enabling developers to reach a wider audience across different platforms (Abramowski, N., 2025).

Integration and Database Connectivity
All three frameworks—Angular, NativeScript, and Ionic—rely on TypeScript, JavaScript, and HTML, ensuring a robust foundation for application development. They are well-suited for integration with various backend technologies, including Node.js, .NET, and Java-based solutions, offering a range of server-side options. For database interactions, Angular applications can connect to various databases through APIs, with popular choices like PostgreSQL and MongoDB often utilized for their scalability and robust features. This versatility allows developers to choose the best tools for their specific project needs, making these frameworks a powerful trio in the modern development ecosystem.

3.1.4 Svelte / Svelte Native

Svelte is a relatively new but rapidly growing framework that takes a unique approach to building user interfaces. Unlike frameworks that do most of their work in the browser, Svelte shifts much of the work to compile time. This means that instead of having a virtual DOM that needs to be updated, Svelte compiles your code into highly optimized, vanilla JavaScript (Abramowski, N., 2025. The result? Faster applications with excellent performance, minimal overhead, and a smaller bundle size. Svelte is also known for its simplicity and ease of use, making it an excellent choice for developers who value a clean and intuitive development experience.

Svelte Native is a framework that extends Svelte's capabilities to native mobile app development. It allows developers to use Svelte's reactive programming model and elegant syntax to build native applications for iOS and Android. By combining the benefits of Svelte's efficiency with native performance, Svelte Native provides a compelling alternative to other cross-platform mobile development solutions. This approach enables developers to create high-performance mobile apps with a streamlined development process, leveraging the same skills and knowledge used for web development.

Svelte and Svelte Native primarily use JavaScript and TypeScript, focusing on efficiency and performance. They are designed to be compatible with any backend technology, including Node.js and Python frameworks, offering flexibility in server-side choices. For database integration, Svelte applications can work with any database through APIs, with Firebase and Django often being preferred due to their ease of use and powerful features.

3.1.5 Flutter

Flutter, a powerful UI toolkit developed by Google that has gained significant popularity for its ability to create natively compiled applications across multiple platforms—mobile, web, and

desktop—using a single codebase. At the heart of Flutter is the Dart programming language, which is celebrated for its performance and ease of use (Abramowski, N., 2025. This combination allows developers to build visually appealing and high-performance applications quickly, Flutter has fast development cycle and expressive UI components. The framework's architecture is designed to provide a seamless experience, enabling developers to focus on crafting beautiful user interfaces without getting bogged down by platform-specific intricacies.

Flutter's adaptability is one of its standout features. It can connect to any backend service, making it suitable for a wide range of projects. Commonly used backend technologies include Node.js and Firebase, which offer robust options for server-side development. For database integration, Flutter works effectively with Firestore, SQLite, and other databases through APIs, ensuring flexible data storage solutions. This versatility allows developers to choose the best tools for their specific needs, making Flutter an excellent choice for modern application development. Whether you're building a mobile app or a web application, Flutter provides a solid foundation for creating high-quality, cross-platform experiences.

3.1.6 SwiftUI / Jetpack Compose

SwiftUI is Apple's declarative framework designed to build UIs across all Apple platforms. It simplifies UI development by using a declarative syntax, where you describe what the UI should look like, and the framework handles the underlying implementation. This approach leads to cleaner, more readable code and makes it easier to create complex interfaces (Abramowski, N., 2025). SwiftUI seamlessly integrates with the Apple ecosystem, providing native performance and a consistent user experience. Its focus on live previews and ease of use makes it a great choice for developers looking to create engaging and performant apps for iOS, macOS, watchOS, and tvOS.

Jetpack Compose is Google's modern toolkit for building native Android UIs. Like SwiftUI, it embraces a declarative approach, simplifying the UI development process. By using Kotlin, Jetpack Compose offers a modern and concise syntax, reducing boilerplate code and improving readability. This framework allows developers to create beautiful, responsive, and efficient user interfaces (Abramowski, N., 2025). It also integrates well with existing Android components and libraries, making it easy to adopt in both new and existing projects. Jetpack Compose provides a streamlined development experience, enabling developers to build high-quality Android apps with ease.

Both SwiftUI and Jetpack Compose excel in integrating with backend services and managing data. They can connect to any backend service, often utilizing Firebase or RESTful APIs for backend communication. For database integration, SwiftUI typically uses Core Data for iOS, while Jetpack Compose can connect to various databases on Android, providing robust data

management capabilities. These frameworks are designed to empower developers to build sophisticated and visually appealing applications with a focus on efficiency, maintainability, and a great user experience.

3.1.7 Frontend Framework Comparison

Criteria	**React.js / React Native**	**Vue.js / NativeScript-Vue / Vue Native**	**Angular / NativeScript / Ionic**
Platform(s)	Web (React.js), Mobile (React Native – iOS & Android)	Web (Vue.js), Mobile (NativeScript-Vue / Vue Native)	Web & Mobile (Cross-platform)
Language(s)	JavaScript, TypeScript	JavaScript, TypeScript, HTML	TypeScript, JavaScript, HTML
Key Features	Component-based architecture; reusable UI blocks; fast virtual DOM; strong ecosystem	Progressive framework; clean, concise syntax; integrated ecosystem (Vue Router, Pinia, Axios)	Full-featured; built-in routing/forms; structured & opinionated; integrates with NativeScript and Ionic
Strengths / Advantages	Fast, dynamic UI; cross-platform reuse; rich library support; strong community	Simple learning curve; lightweight; flexible; rapid frontend development	Enterprise-grade; consistent architecture; strong tooling; native/hybrid deployment options
Ideal Use Cases	Modern, interactive web apps and cross-platform mobile apps	Small to medium web apps; teams prioritizing readability & speed	Large-scale apps; team-based or enterprise projects needing structure & reliability

Integration & Database Connectivity	Works well with Node.js, RESTful APIs; supports MongoDB, Firebase; libraries like Vite/Next.js, Axios, React Query, Redux Toolkit, Context API	Integrates easily with Node.js, Laravel; connects to Firebase, MySQL via APIs	Integrates with Node.js, .NET, Java; connects to PostgreSQL, MongoDB via APIs

Table 2a: Frontend Framework Comparison – Part 1 (Core Information)

Criteria	**Svelte / Svelte Native**	**Flutter**	**SwiftUI / Jetpack Compose**
Platform(s)	Web (Svelte), Mobile (Svelte Native)	Mobile (iOS & Android), Web, Desktop	iOS/macOS/watchOS/tvOS (SwiftUI), Android (Jetpack Compose)
Language(s)	JavaScript, TypeScript	Dart	Swift (SwiftUI), Kotlin (Jetpack Compose)
Key Features	Compile-time rendering (no virtual DOM); reactive syntax; minimal overhead	Single codebase for all platforms; fast UI rendering; rich widgets; hot reload	Declarative UI syntax; live preview; integrates with native APIs
Strengths / Advantages	Very fast apps; small bundle size; clean code; simple to learn	High performance; expressive UI; consistent look across platforms	Native performance; concise syntax; modern UI approach
Ideal Use Cases	Lightweight web or mobile apps needing high performance	Cross-platform apps requiring native-like performance and beautiful UIs	Platform-specific apps needing full native integration and UX

Integration & Database Connectivity	Works with Node.js, Python frameworks; connects to Firebase, Django via APIs	Connects to Node.js, Firebase; supports Firestore, SQLite via APIs	Connects to Firebase or REST APIs; SwiftUI uses Core Data; Jetpack Compose supports Android databases

Table 2b: Frontend Framework Comparison – Part 2 (Core Information)

3.1.8 Progressive Web App (PWA)

A Progressive Web App (PWA) is the simplest and most accessible way to make a web application behave like a mobile app. PWAs run in the browser but can be installed on users' devices, work offline, and send notifications—offering a near-native experience without the need to build a separate mobile application. This approach can be ideal for SplitSync, as it allows the extension of existing web apps for mobile use with minimal extra development. PWAs are lightweight, cost-effective, and require no app store deployment, making them perfect for prototypes or small-scale projects (GeeksforGeeks, 2025).

3.1.9 Core Frontend Framework Choice Summary

After evaluating the listed frameworks, React.js and React Native emerge as the most suitable technologies for building SplitSync. This choice is grounded in their component-based architecture, cross-platform compatibility, and strong developer ecosystem (Abramowski, N., 2025). React.js will enable the development of a responsive and dynamic web version of SplitSync, while React Native allows seamless deployment to both iOS and Android using largely shared code. This reduces development time, ensures consistency across platforms, and provides users with a native-like experience.

React's integration capabilities with Node.js on the backend and RESTful APIs make it ideal for handling SplitSync's functionalities such as real-time expense tracking, receipt scanning, and secure payment processing. Its support for state management libraries (e.g., Redux Toolkit or Context API) ensures efficient data flow and synchronization between users and groups. Additionally, React's broad community support, documentation, and compatibility with tools like Vite and Axios make it a sustainable long-term choice for continuous updates and scalability.

In conclusion, adopting PWA with React.js for web apps offer the best balance between performance, scalability, and cross-platform consistency, aligning perfectly with SplitSync's goal of delivering a smooth, interactive, and unified user experience across devices.

3.2 UI/Styling Frameworks Options

Any of Tailwind CSS, Material UI and Bootstrap could work for this project.

3.2.1 Tailwind CSS

Tailwind CSS is a modern, utility-first CSS framework that allows developers to design directly in the markup using predefined utility classes. It offers exceptional flexibility and speed in building clean, mobile-friendly, and responsive user interfaces without writing extensive custom CSS.

It is compatible with various front-end frameworks, including React, Vue, and Angular, allowing developers to integrate it seamlessly into their projects. Tailwind's utility classes can be applied directly to HTML elements, making it adaptable to any tech stack without the constraints often found in traditional CSS frameworks. This flexibility enables developers to create responsive designs efficiently while maintaining a clean and organized codebase (GeeksforGeeks, 2025).

Tailwind's approach encourages consistency and scalability in styling, making it ideal for creating sleek, professional designs efficiently. For the SplitSync project, it is recommended to combine React with Tailwind CSS (Abramowski, N., 2025).. This combination is widely used in modern web development and strikes an ideal balance between flexibility, performance, and ease of use. React provides a robust structure for building dynamic interfaces, while Tailwind CSS ensures fast styling and a clean, modern aesthetic that adapts seamlessly to various screen sizes.

3.2.2 Material UI

Material UI (MUI) provides a comprehensive set of ready-made React components that implement Google's Material Design principles. It simplifies UI development by offering pre-styled, customizable components such as buttons, forms, dialogs, and navigation bars. MUI ensures a consistent and visually appealing design system while maintaining accessibility and responsiveness, making it a great choice for developers seeking a polished and cohesive interface with minimal design effort.

It is primarily designed for use with React, making it an excellent choice for projects that utilize this library. Material UI provides a comprehensive set of pre-designed components that can be easily customized, allowing developers to create visually appealing applications quickly. While it is primarily focused on React, Material UI can also be integrated with other frameworks through custom setups, although this may require additional configuration (GeeksforGeeks, 2025).

3.2.3 Bootstrap

Bootstrap is one of the most popular and beginner-friendly CSS frameworks, known for its quick prototyping capabilities and ease of use. It comes with a wide range of pre-built components and a responsive grid system, allowing developers to rapidly create layouts that look good across devices.

It is compatible with various front-end technologies, including HTML, CSS, and JavaScript, and can be easily integrated with frameworks like React, Vue, and Angular. Bootstrap's extensive documentation and community support make it a popular choice for developers looking to build responsive and mobile-first applications. Its components can be customized using Sass, allowing for greater flexibility in design (Abramowski, N., 2025).

Bootstrap’s simplicity and extensive documentation make it an excellent option for those new to frontend development or working on smaller-scale projects.

3.2.4 Comparison of UI/Styling Framework Options

Criteria	**Tailwind CSS**	**Material UI (MUI)**	**Bootstrap**
Core Description	A modern, utility-first CSS framework focused on applying styling directly in markup using predefined utility classes.	A React-based UI library implementing Google’s Material Design guidelines.	A classic CSS framework offering pre-built components and a responsive grid system.
Key Features	Utility-first design; responsive, mobile-friendly layouts; highly customizable; clean and scalable code.	Pre-styled, customizable React components (buttons, forms, dialogs, navigation etc.); accessibility-focused; consistent design.	Responsive layout grid; pre-designed UI elements; Sass customization; beginner-friendly.

Strengths / Advantages	Fast to prototype; ensures design consistency; reduces need for custom CSS; boosts developer productivity.	Polished design with minimal effort; strong component system; professional and cohesive UI.	Quick to learn; excellent documentation; easy integration; ideal for rapid prototyping.
Ideal Use Cases	Modern, responsive applications that require flexibility and scalability in design.	React-based apps that need visually consistent, accessible, and modern interfaces.	Small- to medium-scale projects; developers new to frontend frameworks.
Integration / Compatibility	Works seamlessly with React, Vue, Angular, and other frontend stacks.	Primarily built for React; usable with other frameworks with extra setup.	Compatible with HTML, CSS, JavaScript and frameworks like React, Vue, Angular.
Recommendation for SplitSync	Highly recommended — best paired with React for SplitSync; ensures fast styling, clean layout, and modern UI across devices.	Suitable — complements React well but heavier than Tailwind for lightweight projects.	Optional — good for simple or quick prototypes but less flexible than Tailwind for modern UIs.

Table 3: Comparison of UI/Styling Framework Options

3.2.5 UI/Styling Framework Choice Summary

After evaluating the available UI and styling frameworks, Tailwind CSS emerges as the most suitable choice for the SplitSync application. Its utility-first approach provides unmatched flexibility and control over design, allowing developers to create clean, responsive, and modern user interfaces without relying heavily on pre-styled components (GeeksforGeeks, 2025).. Tailwind's emphasis on reusable utility classes encourages consistency and scalability across the project, ensuring that SplitSync maintains a cohesive visual identity as it grows.

Combining Tailwind CSS with React.js offers a particularly effective development workflow. React manages the dynamic and interactive user interface logic, while Tailwind CSS handles layout and design directly within the component markup. This integration streamlines

development, reduces context-switching between CSS files and code, and ensures a mobile-first, responsive experience across devices.

Furthermore, Tailwind CSS is lightweight and performance-optimized, minimizing unnecessary styling overhead while allowing deep customization through its configuration file (Abramowski, N., 2025). This makes it ideal for a modern web application like SplitSync, where a sleek, intuitive, and adaptive interface is essential to enhance user experience.

In conclusion, Tailwind CSS is the recommended styling framework for SplitSync due to its speed, scalability, and perfect alignment with React's component-based architecture, ensuring both efficiency in development and consistency in visual design.

3.3 Backend Framework Options

3.3.1 Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine, allowing developers to execute JavaScript on the server side. It is highly compatible with various front-end frameworks like React, Vue, and Angular, making it a popular choice for full-stack JavaScript development. Node.js is particularly well-suited for building scalable network applications due to its non-blocking, event-driven architecture. It integrates seamlessly with databases such as MongoDB (using Mongoose) and PostgreSQL, and it can also work with RESTful APIs and GraphQL for data management (GeeksforGeeks, 2025).

3.3.2 Django

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It is particularly known for its "batteries-included" philosophy, providing a wide range of built-in features. Django is compatible with various front-end technologies, including React and Vue, through RESTful APIs or GraphQL. It works well with databases like PostgreSQL, MySQL, and SQLite, making it a versatile choice for developers looking to build robust web applications quickly (GeeksforGeeks, 2025).

3.3.3 Ruby on Rails

Ruby on Rails (often simply Rails) is a server-side web application framework written in Ruby. It follows the convention over configuration principle, which allows developers to build applications quickly and efficiently. Rails is compatible with front-end frameworks like React and Vue, and it can be integrated with various databases, including PostgreSQL, MySQL, and SQLite. Its rich ecosystem of gems (libraries) enhances its functionality, making it a popular choice for startups and rapid application development (GeeksforGeeks, 2025).

3.3.4 Spring Boot

Spring Boot is an extension of the Spring framework that simplifies the process of setting up and developing new applications. It is primarily used with Java and is compatible with various front-end frameworks, including Angular and React. Spring Boot can connect to a variety of databases, such as MySQL, PostgreSQL, and MongoDB, and it supports RESTful APIs, making it a robust choice for enterprise-level applications (Navigating Your Tech Stack Choices, 2023).

3.3.5 ASP.NET Core

ASP.NET Core is a cross-platform framework for building modern, cloud-based, internet-connected applications. It is compatible with various front-end frameworks like Angular, React, and Vue. ASP.NET Core can work with multiple databases, including SQL Server, MySQL, and PostgreSQL, and it supports RESTful APIs, making it a strong choice for enterprise applications that require high performance and scalability (GeeksforGeeks, 2025).

3.4 Comparison of Backend Framework Options

Criteria	**Node.js**	**Django**	**Ruby on Rails**	**Spring Boot**	**ASP.NET Core**
Language	JavaScript	Python	Ruby	Java	C# / .NET

Core Description	JavaScript runtime built on Chrome's V8 engine that enables executing JavaScript on the server side.	High-level Python framework following a "batteries-included" philosophy for rapid and secure web development.	Server-side MVC framework emphasizing convention over configuration for faster builds.	Extension of the Spring framework simplifying Java-based application setup and deployment.	Cross-platform framework for modern, cloud-ready web and enterprise applications.
Key Features	Event-driven, non-blocking I/O; scalable; supports REST and GraphQL APIs.	Built-in ORM, authentication, and admin interface; rapid development tools.	Built-in scaffolding; vast gem ecosystem; strong MVC architecture.	Auto-configuration; embedded servers; RESTful API support; enterprise-grade structure.	MVC pattern; dependency injection; RESTful APIs; high performance.
Strengths / Advantages	High performance; full-stack JS compatibility; large community.	Secure and stable; DRY principle; excellent documentation.	Developer-friendly; quick to prototype; productive for startups.	Mature ecosystem; robust and high performance for large systems.	Enterprise-level security; strong Microsoft integration; scalable.
Database Compatibility	MongoDB (via Mongoose), PostgreSQL, MySQL.	PostgreSQL, MySQL, SQLite.	PostgreSQL, MySQL, SQLite.	MySQL, PostgreSQL, MongoDB.	SQL Server, MySQL, PostgreSQL.

Frontend Integration	React, Vue, Angular (seamless full-stack JS integration).	React, Vue (via REST or GraphQL APIs).	React, Vue (via API integration).	React, Angular (via REST APIs).	Angular, React, Vue (via API endpoints).
Best Use Cases / Projects	Real-time apps (chat, payments), scalable APIs, microservices.	Complex web apps requiring security and fast development.	Startup MVPs and fast-iteration projects.	Enterprise-level systems and secure microservice architectures.	Enterprise and cloud-based apps needing scalability and reliability.

Table 4: Databases comparison table (GeeksforGeeks, 2025) and (Abramowski, N., 2025).

Other Backend Stack Options that could also be consider:

- PHP (with frameworks like Laravel, Symfony, or CodeIgniter): PHP remains a popular choice, particularly for content management systems (CMS) and web applications. Laravel is a modern framework that offers a lot of developer-friendly features.
- Go (with frameworks like Gin, Echo, or Beego): Go (Golang) is known for its performance, efficiency, and concurrency features, making it a good choice for building high-performance backend services and APIs.
- Elixir/Phoenix: Elixir is a functional programming language that runs on the Erlang virtual machine. Phoenix is a web framework built on Elixir, known for its speed, scalability, and real-time capabilities.
- Serverless Architectures (using AWS Lambda, Google Cloud Functions, Azure Functions, etc.): These architectures allow you to build backend services without managing servers. They are often used with languages like Node.js, Python, and Go.
- Firebase: A Backend-as-a-Service (BaaS) platform by Google, providing a wide range of services, including real-time databases, authentication, and hosting.

3.5 Framework Choice Summary

After evaluating the available backend frameworks, Node.js emerges as the most suitable choice for the SplitSync application. Its event-driven, non-blocking architecture makes it exceptionally well-suited for building scalable, real-time applications such as expense tracking and group synchronization — both of which are core functionalities of SplitSync.

Node.js enables developers to use JavaScript on both the client and server sides, creating a unified full-stack development environment when paired with React.js on the frontend. This reduces context switching between languages, simplifies debugging, and speeds up overall development. Additionally, Node.js integrates seamlessly with popular databases like MongoDB and PostgreSQL, offering flexible data handling options and strong performance even under concurrent loads.

The extensive NPM (Node Package Manager) ecosystem further strengthens Node.js by providing a wide range of open-source libraries and modules, accelerating feature development without compromising quality (GeeksforGeeks, 2025). For example, Node.js and Firebase work together seamlessly, allowing the build of powerful backend services and leverage Firebase's suite of tools for your applications. This makes it ideal for a student-led project like SplitSync that values efficiency, maintainability, and modern technology alignment.

In conclusion, Node.js is the recommended backend framework for SplitSync due to its high performance, cross-platform compatibility, strong ecosystem, and synergy with React.js, ensuring a consistent and scalable full-stack JavaScript solution.

4.0 Database Options

4.1 Selection criteria (what SplitSync needs)

- Strong consistency & transactions for splits/settlements (no double-charges, accurate balances).
- Flexible querying (groups, expenses, members, time ranges).
- Scales affordably for a student project; easy cloud hosting.
- Real-time updates (optional) for group activity.
- Good ecosystem/ORM support with Node.js
- Binary file support via object storage for receipt images (not the DB).

Table 4.1 shows Database Options Comparison among the databases available.

Criteria	PostgreSQL	MySQL	MongoDB	Firebase Firestore	SQLite	Redis (cache/queues)
Model	Relational (ACID)	Relational (ACID)	Document (JSON)	Document (cloud-managed)	Relational (file-based)	In-memory key/value
Strengths	Rich SQL, strong constraints, JSONB, transactions, extensions; great with Prisma	Fast, familiar, widely hosted; good tooling	Flexible schema; great for nested JSON; horizontal scaling options	Real-time listeners; zero-ops; SDKs; offline on mobile	Zero-setup; tiny footprint; simple	Ultra-fast cache, pub/sub, rate limits, job queues
Trade-offs	Slightly steeper learning than MySQL	Fewer advanced SQL features vs Postgres	Weaker multi-doc transactions; joins are manual	Query limits; pricing by reads/writes; vendor lock-in	No concurrency for web scale; no server features	Not a system of record; data is volatile (unless Redis persistence configured)
Best for	Financial-like records, integrity, transactional updates, analytics	Traditional CRUD apps with simple joins	Flexible content, logs, denormalized reads	Real-time feeds, prototypes, mobile sync	Local dev, offline tools, demos	Caching hot queries, notifications, queues
Node.js/ORM	Excellent (Prisma/TypeORM/Knex)	Excellent	Good (Mongoose, Prisma)	SDKs (no SQL/ORM)	Good for dev tooling	Clients for Node (bullmq, ioredis)

Hosting	Supabase, Neon, Railway, Render, AWS RDS	PlanetScale, AWS RDS, Railway, Render	Atlas, Railway, Render	Firebase (GCP)	On device/server file	Upstash, Redis Cloud, self-host

Table 5: Database Options Comparison

5.0 Payment Gateway Integration

In app development, a payment gateway is a service that securely processes electronic payments, enabling mobile applications to accept credit cards, debit cards, and digital wallet transactions from users. It acts as a digital bridge between the app, the customer, and the financial institutions involved to ensure that sensitive payment information is handled safely and the transaction is completed successfully (Oleg, 2025). The key considerations for selecting a payment gateway include:

- In-app vs. tangible purchases: App stores like Apple's App Store and Google Play have specific rules regarding payment processing. If an app sells digital content (e.g., premium features, content downloads), it is often required to use the platform's native in-app purchase system. If it sells physical goods or services, it will integrate a third-party payment gateway.

- Security and compliance: It is critical to ensure the gateway is PCI DSS (Payment Card Industry Data Security Standard) compliant to protect sensitive customer data and avoid security risks. Many gateway SDKs handle encryption and tokenization to simplify compliance for the developer.

- User experience: A smooth, fast, and reliable checkout process reduces cart abandonment and increases customer trust. Features like support for multiple payment methods, mobile-optimized interfaces, and the use of native platform features like Apple Pay and Google Pay all contribute to a better user experience.

5.1 How different payment gateway differ

Extracted from Drozdovica, J. (2024), Gottumukkala, M. (2025), and Levi, G (2025), the broad categories of payment gateway include;

- By Integration Method
    1. Hosted Payment Gateways (Off-Site/Redirect) : Customers are redirected to the payment provider's page to complete transactions. Examples include PayPal Standard, Stripe Checkout, and 2Checkout. These are easiest to implement and handle PCI compliance, but offer less control over the user experience.
    2. Self-Hosted Payment Gateways (On-Site): The entire payment process happens on a website, with payment data submitted directly to the gateway. This provides full control over design and user experience but requires handling PCI DSS compliance. Examples include Authorize.Net and some Stripe implementations.
    3. API/Integrated Payment Gateways: These use APIs to integrate payment processing directly into a platform while keeping customers on site. The gateway handles security in the background Gottumukkala, M. (2025). Examples include Stripe API, Braintree, and Square. They balance customization with security.

- By Business Model
    1. Direct Payment Gateways: Established by banks or financial institutions, these connect directly to merchant accounts. Examples include Chase Paymentech and Wells Fargo Payment Gateway. They typically offer better rates for high-volume businesses.
    2. Payment Aggregators/Third-Party Gateways: These pool multiple merchants under one master merchant account. Examples include PayPal, Stripe, and Square. They're faster to set up and ideal for small businesses or startups, though fees may be higher.
    3. Platform-Based Payment Gateways: Built specifically for e-commerce platforms like Shopify Payments for Shopify or WooCommerce Payments for WordPress.

- By Payment Channel
    1. Online Payment Gateways - For e-commerce and web-based transactions
    2. Mobile Payment Gateways - Optimized for in-app and mobile web payments
    3. Point-of-Sale (POS) Gateways - For in-person retail transactions
    4. Omnichannel Gateways - Support multiple channels (online, mobile, in-store) through one platform

5.2 How payment gateways work in mobile apps

- Initiating payment: The user begins the checkout process within the mobile app.

- Sending information: The app securely sends the encrypted payment details to the payment gateway. Many providers offer Software Development Kits (SDKs) to simplify this process and handle the encryption.

- Authorization: The payment gateway sends an authorization request to the user's bank or card network to verify funds and approve the transaction.

- Transaction completion: The bank approves or declines the request, and the status is sent back through the payment gateway to the app. The funds are then transferred to the merchant's account.

5.3 Popular payment gateway provider options.

As highlighted by Oleg (2025) and Sharma (2007), familiar payment gateways known include:

- Stripe: A developer-friendly platform with robust APIs and tools for handling subscriptions, fraud detection, and multi-vendor marketplaces.

- PayPal/Braintree: Braintree offers a comprehensive SDK for iOS and Android, which allows for in-app payments using credit cards, digital wallets, and Venmo. PayPal is also a well-known option for easy integration and user trust.

- Apple Pay and Google Pay: These are mobile-native payment solutions that allow users to pay with information already stored in their Apple or Google accounts. They offer a fast, convenient, and secure checkout experience that can be integrated using their respective APIs and SDKs.

- Razorpay is an emerging player in the payment gateway space, especially popular in India. It supports multiple payment methods, including UPI, wallets, and cards, with a seamless checkout experience.

- Square: An all-in-one payment solution well-suited for businesses with both online and offline sales.

- Braintree: An API-driven payment gateway providing various payment support, from credit cards to digital wallets. It is a subsidiary of PayPal, which is primarily focused on the mobile payment market, and serves many large market players such as Uber and AirBnB. This service can be called the most progressive, and those who already use it emphasize their good technical support that responds to requests very quickly. The process of the payment gateway integration into a mobile app is quite simple; the creators

of Braintree have managed to make it so that just a few lines of code need to be written (Lvivity, 2024).

- Authorize.Net: A self-hosted traditional payment processor that ensures secure credit card processing.

- WorldPay: Primarily works with local banks to process payments.

Table 5.2a and 5.2b shows the features, integration, and market focus of the payment gateways.

Feature / Gateway	**Stripe**	**PayPal / Braintree**	**Apple Pay**	**Google Pay**
Type	Developer-friendly platform with robust APIs	PayPal: Trusted brand, easy integration; Braintree: API-driven, mobile SDKs	Mobile-native payment solution	Mobile-native payment solution
API / SDK	Extensive APIs and developer tools	Braintree: Comprehensive SDK for iOS/Android; PayPal: simple integration	API and SDK for iOS devices	API and SDK for Android devices
Payment Methods	Credit cards, digital wallets, subscriptions, multi-vendor marketplaces	Credit cards, digital wallets, Venmo	Apple account stored cards	Google account stored cards
Market Focus	Global, developer-centric	Global, mobile payments focus (Braintree)	Mobile users on Apple devices	Mobile users on Android devices
Integration Complexity	Moderate to advanced,	Simple for PayPal; Braintree	Moderate, requires Apple ecosystem	Moderate, requires Google ecosystem

	developer-friendly	requires few lines of code		
Security Features	Fraud detection, PCI compliance	Fraud detection, PCI compliance	Secure tokenization	Secure tokenization
Notable Clients	Many startups and large companies	Uber, Airbnb (Braintree)	Apple device users	Android device users
Unique Selling Point	Robust APIs for subscriptions and marketplaces	Fast technical support, mobile SD	Fast, convenient mobile payments.	Fast, convenient mobile payments.

Table 5a Features, Integration, and Market Focus

Feature / Gateway	**Razorpay**	**Square**	**Authorize.Net**	**WorldPay**
Type	Emerging payment gateway in India	All-in-one online & offline payment	Traditional self-hosted processor	Works with local banks
API / SDK	APIs supporting UPI, wallets, cards	APIs and SDKs for POS and online	API for secure credit card processing	API for bank-based payment processing
Payment Methods	UPI, wallets, cards	Credit cards, digital wallets, POS	Credit cards	Credit cards, bank transfers
Market Focus	India-focused, growing rapidly	Small to medium businesses, retail	Traditional merchants, secure processing	Local banks and merchants

Integration Complexity	Moderate, seamless checkout experience	Easy for online and offline	Moderate, self-hosted setup	Moderate, bank integration required
Security Features	PCI compliant, secure checkout	PCI compliant, fraud prevention	Secure credit card processing	Secure bank payment processing
Notable Clients	Popular in Indian market	Retailers and SMBs	Traditional merchants	Local merchants and banks
Unique Selling Point	Strong Indian payment ecosystem	Unified online/offline sales solution	Reliable traditional processor	Strong local bank partnerships
Pricing (Typical Fees)	Standard plan: ~2% per transaction; Enterprise plan custom pricing	In-person: 1.75% + VAT; Online: 2.6%-3.5% + fixed fee	Starts at $25/month + 2.9% + $0.30 per transaction	Monthly fee ~£19.95 + transaction fees (e.g., 1.3% + 20p for Visa/Mastercard

Table 5b Features, Integration, and Market Focus

6.0 Security and Safety for Fintech Applications

Security represents the cornerstone of any financial technology application, particularly for expense-sharing platforms that handle sensitive user data and financial transactions (Mahesh, H., 2025). The Split-Sync application requires a multi-layered security approach encompassing authentication, encryption, payment compliance, and API protection to ensure user trust and regulatory compliance.

6.1 Authentication and identity management

Modern fintech applications have evolved beyond traditional password-based authentication to embrace more robust identity verification mechanisms. The authentication landscape for expense-sharing applications centres on three primary approaches: OAuth 2.0, JSON Web Tokens (JWT), and multi-factor authentication (MFA). OAuth 2.0 functions as an authorization framework particularly well-suited for third-party integrations, enabling users to authenticate through trusted providers such as Google or Facebook without exposing credentials to the

application itself. Research from Strapi and Frontegg in 2024 demonstrates that combining OAuth 2.0 flows with JWT tokens creates a powerful hybrid approach, where OAuth handles initial authorization while JWT provides stateless, self-contained tokens for subsequent requests.

Multi-factor authentication has emerged as a critical security layer, with effectiveness studies showing a 99.9% reduction in unauthorized access when properly implemented. SuperToken (2025) Contemporary best practices favour app-based authenticators such as Duo or Authy over SMS-based verification due to SIM-swapping vulnerabilities that have compromised SMS-based systems. Andersen, G. (2025) mentioned that Fintech leaders including Venmo, Zelle, and Razorpay have adopted MFA not only for initial login but also for sensitive operations such as large transactions, new device authorization, and password resets.

For the React and Firebase technology stack, Firebase Authentication provides an integrated solution that seamlessly combines OAuth 2.0 with major identity providers, delivers JWT tokens automatically, and supports phone-based MFA. The service offers remarkable value for student projects with a generous free tier supporting up to 50,000 monthly active users, with phone authentication costs of merely $0.01 per verification after the initial 10,000 free verifications monthly. AWS (2025) pointed out that this pricing structure makes enterprise-grade authentication accessible to student developers while maintaining production-ready security standards.

6.2 Data encryption and protection

Data encryption operates at multiple levels within modern fintech applications, protecting information both at rest and in transit. The current industry standard for data at rest employs AES-256 encryption, which governments, military organizations, and financial institutions universally recognize as the gold standard. For data in transit, Transport Layer Security - TLS 1.3 represents the minimum acceptable standard as of 2025, though TLS 1.2 remains acceptable for legacy system support. Team W. C., (2025) stated that all HTTP communications must enforce HTTPS to prevent man-in-the-middle attacks and ensure data integrity during transmission.

End-to-end encryption for sensitive financial data typically employs a hybrid approach, utilizing RSA or Elliptic Curve Cryptography for secure key exchange combined with AES for payload encryption. This methodology balances computational efficiency with robust security. Research by Olaiya and colleagues published in ResearchGate during 2024 emphasizes that key management represents the most critical aspect of any encryption strategy. Mundra, P. (2025) warned that modern applications should never hardcode encryption keys within source code; instead, they should leverage dedicated key management services such as AWS Key Management Service, Azure Key Vault, or HashiCorp Vault. (Team, D., 2025)

Field-level encryption provides an additional protection layer, encrypting individual sensitive fields such as social security numbers or credit card numbers separately within database records. According to Victor (2025), this approach minimizes exposure if database access is compromised, as attackers would need both database access and encryption keys to decrypt sensitive information. Tokenization offers an alternative strategy where sensitive data is replaced with non-sensitive tokens, with companies like Stripe, VGS, and AWS Payment Cryptography providing tokenization services. Schefflier (2025) mentioned that Cash App serves as a notable implementation example, employing end-to-end encryption across all system layers, significantly reducing insider breach risks while maintaining system functionality. Medium

6.3 PCI DSS compliance requirements

The Payment Card Industry Data Security Standard (PCI DSS) version 4.0.1 establishes twelve comprehensive requirements that organizations handling credit card information must address (Watson, 2025) . These requirements encompass network security controls, secure system configurations, protection of stored account data, strong cryptography for data transmission, malware protection, secure systems and software development, access restrictions based on need-to-know principles, user identification and authentication, physical access restrictions, comprehensive logging and monitoring, regular security testing, and documented information security policies (Wikipedia., 2025 & Iyer. S., 2025).

Organizations fall into different compliance levels based on transaction volume, with Level 4 encompassing businesses processing fewer than 20,000 transactions annually and typically requiring only a Self-Assessment Questionnaire. (Wikipedia., 2025) Student projects generally fall within this category, making compliance more manageable though no less important. Non-compliance carries substantial penalties ranging from $5,000 to $100,000 monthly, while the average cost of a payment card data breach reached $5.9 million according to IBM's 2023 research. (Iyer. S., 2025) These figures underscore the financial imperative of proper security implementation.

For student developers, the most pragmatic approach to PCI compliance involves leveraging PCI-compliant payment processors such as Stripe or PayPal. This strategy significantly reduces the compliance scope by ensuring that sensitive payment data never touches the application's servers. (StripeDocs.,2025) Stripe, certified as a PCI Level 1 Service Provider, provides comprehensive tokenization, fraud detection through Stripe Radar, 3D Secure authentication, and automated compliance maintenance. (StripeDocs., 2025) By implementing Stripe Elements or Checkout, developers can accept payments while maintaining PCI compliance without requiring extensive security infrastructure investment(StripeDocs., 2025).

6.4 API security and protection mechanisms

Application Programming Interface security forms a critical component of fintech application architecture, particularly for expense-sharing platforms that expose multiple endpoints for data manipulation. (Iyer. S., 2025) The OWASP API Security Top 10 for 2023 identifies unrestricted resource consumption as a primary vulnerability, emphasizing the necessity of rate limiting to prevent denial-of-service attacks and resource exhaustion. Rate limiting should operate at multiple levels: user-based limits for authenticated requests typically range from 100 to 1,000 requests per hour, while IP-based limits for pre-authentication endpoints such as login should be more restrictive at 3 to 10 requests per minute to prevent brute-force attacks (OWASP, 2025).

JWT token management requires careful consideration of security implications. Access tokens should maintain short lifespans between 15 and 60 minutes to limit exposure if compromised, while refresh tokens can persist for days or weeks but should implement rotation to minimize long-term vulnerability. Storage mechanisms for tokens critically impact security; tokens should be stored in HTTP-only cookies rather than localStorage to mitigate Cross-Site Scripting (XSS) vulnerabilities. Token signing algorithms such as HS256 or RS256 ensure token integrity, while including expiration timestamps in the token claims enables automatic token invalidation.

6.5 Real-world implementations and Firebase security

Contemporary expense-sharing platforms demonstrate varied approaches to security implementation. Splitwise's integration with Venmo showcases OAuth-based cross-platform authentication, (Wikipedia., 2025) though security researchers have identified historical vulnerabilities including a 2FA bug on the Splitwise website that persisted for months after initial discovery in June 2017, and email mismatch issues causing payment routing failures. Splitwise Pay implements rigorous identity verification requirements including phone verification, legal name, address, date of birth, and social security number to satisfy federal compliance mandates before enabling account linking.

Firebase security for the React and Node.js technology stack requires meticulous attention to Firestore Security Rules. Best practices dictate that developers should begin with default-deny rules, explicitly granting permissions only where necessary. User-based access control ensures that individuals can only access their own data through rules such as `allow write: if request.auth.uid == uid`. Production deployments must avoid wildcard permissions and global `true` values that would expose all data. The Firebase Emulator Suite enables comprehensive security rule testing before deployment, allowing developers to identify vulnerabilities in a safe development environment. Field-level validation within security rules ensures data integrity by verifying that incoming data matches expected schemas and value ranges (firebase, 2025).

For student projects, Firebase offers exceptional value with costs typically ranging from zero to five dollars monthly for applications supporting 1,000 to 5,000 users. The free tier includes 50,000 monthly active users for authentication, 1GB Firestore storage with 50,000 daily reads, 1GB hosting storage with 10GB monthly transfer, and 2 million monthly Cloud Functions invocations. This generous allocation enables students to build production-quality applications while learning enterprise security practices without financial burden (AWS, 2025).

7.0 Hosting and Deployment Infrastructure

The hosting and deployment infrastructure for modern web applications has evolved dramatically, with Platform-as-a-Service offerings enabling rapid deployment and scalability without extensive DevOps expertise. For student developers building expense-sharing applications with React frontends and Node.js or Firebase backends, platform selection significantly impacts development velocity, operational costs, and long-term scalability.

7.1 Platform comparison and selection criteria

Contemporary hosting platforms offer varying strengths tailored to different use cases and technical stacks. Vercel has emerged as the premier platform for frontend deployment, particularly for React applications, offering zero-configuration deployment with automatic HTTPS provisioning, global Content Delivery Network distribution, and preview deployments for every pull request. The platform's free Hobby tier provides 100GB monthly bandwidth, unlimited deployments, and one million edge requests, making it exceptionally suitable for student projects (Ayres, 2019 & Hoang, C., 2025). Vercel's serverless functions enable backend logic execution without dedicated server management, though the platform primarily focuses on frontend hosting with backend capabilities limited to serverless architectures.

Firebase Hosting integrates seamlessly with other Firebase services, providing a compelling option for applications built entirely within the Firebase ecosystem. According to Lohar (2024) the free Spark plan offers 10GB storage and approximately 10.5GB monthly transfer (360MB daily), with SSL certificates included automatically. (Hoang, C., 2025) Firebase's true strength emerges from its complete backend solution encompassing authentication, Firestore database, Cloud Functions, and file storage, all available within generous free tiers. This integrated approach simplifies development by eliminating the need to coordinate multiple service providers and manage inter-service communication.

Railway represents a compelling alternative for developers requiring traditional Node.js backends with full database hosting. Following Heroku's discontinuation of free tiers in November 2022, Railway has positioned itself as the spiritual successor, offering usage-based pricing with a $5 monthly credit included in the $5 monthly subscription. Real-world cost

analysis demonstrates that small applications frequently consume less than the included credits, effectively resulting in zero cost beyond the base subscription. Railway provides one-click deployment for PostgreSQL, MySQL, MongoDB, and Redis, with Docker support enabling flexible deployment configurations. The platform's modern interface and straightforward pricing model make it particularly attractive for student developers learning full-stack development.

AWS Amplify serves developers seeking to build AWS expertise while benefiting from comprehensive cloud infrastructure. Starting July 2025, AWS introduced up to $200 in credits for six months specifically targeting student developers, complementing the standard 12-month free tier offering 1,000 build minutes and 15GB monthly transfer. Amplify integrates with numerous AWS services including Lambda for serverless functions, DynamoDB for NoSQL storage, Cognito for authentication, and AppSync for GraphQL APIs. However, this comprehensive capability comes with increased complexity and a steeper learning curve, making it more suitable for students prioritizing AWS skill development over rapid application deployment.

7.2 CI/CD pipeline implementation

Continuous Integration and Continuous Deployment pipelines automate testing and deployment processes, ensuring code quality and reducing manual deployment errors. GitHub Actions has emerged as the dominant CI/CD solution for projects hosted on GitHub, offering seamless integration with repository workflows and generous free tier allocation for public repositories. (Hoang, C., 2025) A comprehensive CI/CD pipeline for an expense-sharing application should encompass multiple stages: dependency installation using `npm ci` to ensure reproducible builds, linting to enforce code style standards, unit test execution with coverage reporting, end-to-end test execution to validate user workflows, security audits using `npm audit`, and finally automated deployment to the hosting platform upon successful test completion.

Vercel's native CI/CD integration provides the simplest deployment experience, automatically detecting repository changes and triggering deployments without additional configuration. Each commit to the main branch triggers a production deployment, while pull requests generate preview deployments with unique URLs for testing before merge. This workflow enables effective collaboration by allowing team members to review functional changes alongside code changes (Robinson, 2025). Firebase deployment through GitHub Actions requires more explicit configuration but offers greater control over deployment timing and conditions, enabling strategies such as scheduled deployments or manual approval gates for production releases (React., 2025).

Railway provides automatic deployment from GitHub repositories similar to Vercel, detecting changes and rebuilding containers automatically. The platform's Docker support enables sophisticated deployment strategies including multi-stage builds for optimized container sizes

and separate development and production configurations. For teams requiring additional deployment control, explicit GitHub Actions workflows can orchestrate testing, building, and deployment phases while providing detailed logs and deployment status notifications through Slack or other communication platforms.

7.3 Performance and cost considerations

Performance benchmarks reveal significant variations among hosting platforms, though differences for small-scale applications often prove negligible in practice. Vercel demonstrates the fastest Time to First Byte typically under 100 milliseconds and Largest Contentful Paint under 1.5 seconds for React single-page applications, attributable to its edge network optimization and sophisticated caching strategies. FlightFormation Firebase Hosting delivers comparable performance at slightly higher latencies around 150 milliseconds TTFB, leveraging Google's global CDN infrastructure. (Hoang, C., 2025) Railway exhibits marginally higher latencies near 200 milliseconds TTFB due to regional rather than global edge distribution, though cold start times of approximately 200 milliseconds dramatically outperform the 30-second cold starts that plagued Heroku's free tier.

Cost projections demonstrate the economic viability of modern hosting platforms for student projects. Applications serving 100 to 500 users typically incur zero costs across all major platforms, with free tiers providing sufficient resources. Scaling to 5,000 users generally requires $10-40 monthly expenditure on Vercel, $10-20 monthly on Firebase, $15-30 monthly on Railway, and $30-50 monthly on AWS Amplify. Qovery This cost structure enables students to launch applications with confidence that hosting expenses will remain manageable even with moderate success. The Vercel and Firebase combination recommended for Split-Sync offers optimal cost efficiency, with both platforms providing generous free tiers sufficient for applications serving 500 to 1,000 active users without any financial investment.

7.4 Deployment architecture and environment management

The recommended architecture for Split-Sync employs Vercel for React frontend hosting and Firebase for backend services including Firestore database, authentication, Cloud Functions, and file storage. This separation of concerns enables independent scaling and optimization of frontend and backend components. The frontend deployment to Vercel provides global CDN distribution, automatic SSL certificate management, and HTTP/2 support, while Firebase's globally distributed infrastructure ensures low-latency database access from any geographic region.

Environment variable management requires careful attention to security, particularly for fintech applications handling sensitive configuration. Vercel provides both dashboard-based and CLI-based environment variable configuration, with variables segregated across Production,

Preview, and Development environments to prevent accidental credential exposure. Newline Variables should be encrypted at rest and transmitted securely, with secrets rotation occurring quarterly at minimum. Firebase offers environment configuration through the Functions Config service for Cloud Functions, while React applications should leverage build-time environment variables prefixed with `REACT_APP_` to ensure client-side availability while maintaining security boundaries. Create React App

8.0 OCR Receipt Scanning Technologies

Optical Character Recognition technology enables expense-sharing applications to extract structured data from receipt images, dramatically reducing manual data entry and improving user experience. The technology landscape encompasses open-source solutions, cloud-based APIs, and specialized receipt processing services, each offering distinct trade-offs among accuracy, implementation complexity, feature richness, and cost.

8.1 Technology landscape and accuracy comparison

Tesseract.js represents the open-source standard for browser-based OCR, providing JavaScript and WebAssembly implementations of Google's Tesseract engine. Artem SapeginfreeCodeCamp The technology delivers highly variable accuracy ranging from 85-95% on high-quality scans to 50-75% on typical mobile photos and as low as 20-50% on poor-quality or crumpled receipts. MediumAIMultiple This performance variability stems from Tesseract's reliance on preprocessing quality, with optimal results requiring extensive image enhancement including resolution scaling to 300 DPI, grayscale conversion, Gaussian blur for noise reduction, binarization using Otsu's method, deskewing through angle detection, and contrast adjustment. DocsumoMedium Implementation complexity proves substantial, with basic implementations requiring 100-200 lines of code but production-ready systems demanding 500+ lines alongside 16-40 hours of preprocessing pipeline development.

Google Cloud Vision API delivers consistent 98% overall accuracy on receipt text extraction according to AIMultiple's March 2025 benchmark study of 30 diverse documents. The service demonstrates 99.2% accuracy on printed text and 82-90% accuracy on handwritten content, with receipt-specific performance ranging from 95-98%. aimultiple +2 Implementation complexity remains minimal, requiring merely 50-100 lines of code and 2-4 hours setup time due to built-in preprocessing and automatic language detection across 60+ languages. The API returns text with bounding box coordinates for each detected word, Google Cloud though it provides unstructured output requiring custom parsing logic to extract line-item pairings of products and prices.

AWS Textract achieves accuracy rates between 98-99.3% overall, with its specialized AnalyzeExpense API delivering 95-97% accuracy on receipt-specific structured extraction. AIMultiple. (2025) stated that extract's distinguishing feature lies in its sophisticated understanding of receipt semantics, automatically normalizing field variations such as recognizing "invoice no", "receipt #", and "invoice number" as equivalent INVOICE_RECEIPT_ID fields. The service automatically pairs line items with quantities and prices, distinguishes between vendor, supplier, bill-to, and ship-to addresses, and infers missing information such as locating vendor names within logos when not explicitly printed as text. (AWS., 2025) Implementation requires 75-125 lines of code with 3-6 hours setup time including AWS infrastructure configuration, resulting in structured JSON output significantly reducing post-processing requirements compared to plain text OCR.

8.2 Implementation complexity and integration patterns

The development effort required to achieve production-ready receipt scanning varies dramatically across technologies. Tesseract.js demands substantial investment with 52-112 hours to minimum viable product, comprising 4-8 hours OCR integration, 16-40 hours preprocessing pipeline development, 24-48 hours parsing logic implementation for extracting structured data from unstructured text, and 8-16 hours error handling. This substantial investment stems from the necessity of building receipt-specific parsing logic to identify line items, match prices with products, extract totals, and handle multi-line item descriptions. Regular expressions and heuristics-based parsing prove fragile across diverse receipt formats, requiring continuous refinement as users encounter receipts from new merchants.

Google Vision API reduces implementation burden to 22-36 hours total, eliminating preprocessing requirements entirely while still necessitating 16-24 hours developing parsing logic to structure the returned text into meaningful receipt components. The service handles variable image quality, orientations, and lighting conditions automatically, freeing developers to focus on business logic rather than image processing. Integration with React frontends and Firebase storage follows a straightforward pattern: users upload images to Firebase Storage, triggering a Cloud Function that calls Vision API, parses results into structured data, and stores the structured receipt in Firestore for display in the application.

AWS Textract's AnalyzeExpense API offers the fastest path to production at 15-30 hours implementation time, with automatic structuring substantially reducing parsing logic requirements to 8-16 hours. The service returns SummaryFields containing vendor name, date, total, and tax alongside LineItemGroups containing arrays of line items with paired products, quantities, and prices. (AWS., 2025) This structured output enables direct storage in databases with minimal transformation, accelerating development velocity and reducing maintenance burden as receipt format variations are handled by AWS's continuously improving machine learning models rather than custom application code.

8.3 Cost analysis and recommendations

Financial considerations for receipt OCR vary dramatically based on processing volume and chosen technology. Tesseract.js imposes zero direct costs as an open-source solution, though this apparent economy proves misleading when accounting for development time and infrastructure. Assuming student developer labor valued at $15 hourly, the 60-hour additional development effort compared to cloud solutions represents $900 in opportunity cost. Furthermore, Tesseract requires self-hosted compute resources for server-side processing, adding $50-200 monthly infrastructure costs for handling 100,000 receipts, negating cost advantages at scale.

Google Vision API employs straightforward pricing with the first 1,000 images monthly completely free, followed by $1.50 per 1,000 images ($0.0015 per image) for volumes between 1,001 and 5,000,000 monthly. googleGoogle Cloud Cost examples illustrate the service's affordability: 100 receipts monthly cost zero dollars, 1,000 receipts monthly cost zero dollars, 10,000 receipts monthly cost $13.50, and 100,000 receipts monthly cost $148.50. For student projects processing fewer than 1,000 receipts monthly, Vision API remains entirely free throughout development and initial deployment, making it the optimal choice for learning and prototyping.

AWS Textract AnalyzeExpense pricing begins with a three-month free tier offering 100 pages monthly, transitioning to $0.01 per page for the first million pages monthly (AWS., 2025) While this pricing appears competitive at small scales, costs escalate rapidly: 10,000 monthly receipts cost $100, and 100,000 monthly receipts cost $1,000. The substantially higher cost compared to Vision API positions Textract as optimal for enterprise applications where the value of structured extraction justifies premium pricing, or for applications so receipt-heavy that the automatic structuring saves enough development time to offset ongoing API costs.

Total cost of ownership analysis over the first year for an application processing 10,000 receipts monthly reveals clear winners. Tesseract.js totals $10,650-11,850 encompassing $7,875 development, $1,200-2,400 annual hosting, and $1,575 annual maintenance. Google Vision API totals merely $3,402 comprising $2,700 development and $162 annual API costs. AWS Textract totals $4,050 including $2,250 development and $1,200 annual API costs. These calculations demonstrate Google Vision API's superior cost-effectiveness until approximately 50,000 monthly receipts, at which point the cumulative API costs potentially justify investment in self-hosted Tesseract infrastructure.

8.4 Technology recommendations for Split-Sync

For the Split-Sync student project, Google Cloud Vision API emerges as the optimal choice, balancing high accuracy, rapid implementation, and zero cost during development and early deployment. The free tier of 1,000 monthly receipts provides ample capacity for user testing and

initial launch, while the straightforward pricing model ensures predictable costs should usage exceed free tier limits. Implementation should follow a serverless architecture with images uploaded to Firebase Storage triggering Cloud Functions that invoke Vision API, parse returned text to extract merchant name, date, total amount, and optionally line items, and store structured data in Firestore. This architecture maintains PCI compliance by avoiding storage of payment card images while enabling receipt-based expense documentation.

Future enhancement opportunities include implementing preprocessing for particularly challenging receipt images using client-side JavaScript libraries such as Sharp or OpenCV.js, developing machine learning models to improve merchant name recognition and categorization, and implementing confidence thresholds where low-confidence OCR results prompt user review before saving. (AWS., 2025) For receipts where automatic extraction fails, the application should provide intuitive manual entry interfaces with smart defaults populated from partially successful OCR results, ensuring graceful degradation that maintains user experience even when automated systems underperform.

9.0 State Management Solutions for React Applications

State management represents a foundational architectural decision for React applications, influencing code organization, application performance, developer experience, and long-term maintainability. Expense-sharing applications present particular state management challenges including real-time synchronization across multiple devices, optimistic updates for responsive user interfaces, complex derived state such as balance calculations, and integration with Firebase Firestore's real-time listeners. Three primary approaches merit consideration: React's built-in Context API, Redux Toolkit as the evolution of Redux, and Zustand as a modern lightweight alternative.

9.1 Context API: React's native solution

React's Context API provides state management capabilities without external dependencies, making it attractive for applications with modest state requirements. The API excels at managing relatively static global state such as user authentication status, theme preferences, internationalization settings, and UI configuration. dev Implementation requires minimal boilerplate, with developers creating contexts using `createContext`, providing state through `Provider` components, and consuming state via `useContext` hooks. For small applications with fewer than 50 components, Context API often proves sufficient and appropriate. (Dev. 2025).

However, Context API demonstrates significant performance limitations that become problematic in applications with frequently updating state. All components consuming a context re-render whenever any value within that context changes, regardless of whether the component actually uses the changed value. dev This limitation cannot be fully mitigated through `useMemo` optimization, as React's context mechanism fundamentally lacks selector-based subscriptions. Performance benchmarks demonstrate average render times of 350 milliseconds for single large contexts managing 1,000 components, improving to 120 milliseconds when contexts are split by domain, and 75 milliseconds for update operations. (Dev. 2025) These latencies prove acceptable for infrequent updates but create noticeable lag for real-time expense tracking where multiple users may add or modify expenses simultaneously.

Firebase integration with Context API follows straightforward patterns with Firestore's `onSnapshot` listener established within a context provider's `useEffect` hook, updating context state on each document or collection change. Stack Overflow This implementation works adequately for read-heavy applications but struggles with complex write patterns requiring optimistic updates and rollback capabilities. The absence of built-in development tools for debugging context state proves challenging for troubleshooting production issues, as developers must rely on React DevTools' generic component tree without specialized state inspection capabilities.

9.2 Redux Toolkit: Enterprise-grade state management

Redux Toolkit emerged as the official recommended approach for Redux implementation, addressing historical criticisms of Redux's verbosity and complexity through opinionated defaults and utility functions. ReduxRedux The toolkit significantly reduces boilerplate through `createSlice` combining reducers and actions, `createAsyncThunk` standardizing async logic patterns, and `configureStore` providing sensible defaults including Redux DevTools integration and thunk middleware. Prakashinfotech +2 RTK Query additionally provides powerful data fetching and caching capabilities, though Firebase applications typically manage data synchronization through Firestore listeners rather than REST API calls.

Redux Toolkit targets large-scale applications with complex state interactions, particularly those requiring time-travel debugging, extensive middleware chains for logging and analytics, or strict separation between business logic and presentation components. (Dev. 2025) The architecture proves valuable for teams exceeding 15 developers across multiple teams, where standardized patterns and predictable state mutations facilitate coordination and reduce merge conflicts. Redux DevTools provide exceptional debugging capabilities including action replay, state diffing, and comprehensive action logging, making them invaluable for diagnosing production issues in complex applications.

Performance characteristics of Redux Toolkit show initial render times averaging 210 milliseconds with frequent small updates requiring 65 milliseconds, improving to 45 milliseconds when leveraging entity adapters for normalized state and memoized selectors for derived state computation. Bundle size adds approximately 15 kilobytes after minification and compression, representing meaningful overhead for applications where minimal bundle size proves critical. dev Learning curve remains substantial despite Redux Toolkit's improvements, with developers requiring 2-3 weeks to achieve productivity when unfamiliar with Redux's conceptual model of actions, reducers, and unidirectional data flow.

Firebase integration with Redux Toolkit requires explicit middleware or thunks to establish Firestore listeners and dispatch actions updating Redux state. Optimistic updates demand careful coordination between local Redux state updates and eventual Firestore synchronization, with rollback logic implemented through Redux actions if Firestore writes fail. This complexity proves manageable for experienced Redux developers but presents substantial cognitive overhead for teams learning Redux and Firebase simultaneously. The architecture's value emerges primarily in applications where complex business logic requires substantial derived state computation, multi-step workflows demand coordinated state updates across numerous components, or regulatory requirements necessitate comprehensive audit logging of all state mutations.

9.3 Zustand: Lightweight modern alternative

Zustand positions itself as a minimalist state management solution with a hooks-based API, delivering Redux-like capabilities with dramatically reduced boilerplate and bundle size. (Dev. 2025) The library gained substantial traction with over 30,000 GitHub stars, establishing itself as a mature production-ready solution. Prakashinfotech +2 Zustand's core API proves remarkably simple, with stores created through a single `create` function accepting a setter function that returns the initial state and action methods. Zustand Components consume state by calling the store hook with optional selectors enabling fine-grained subscriptions to specific state slices, preventing unnecessary re-renders when unrelated state changes. subhojit +2

Performance benchmarks position Zustand as the fastest option with initial render times averaging 160 milliseconds and update times of merely 35 milliseconds for frequent small changes, improving further to 18 milliseconds when employing fine-grained selectors. Bundle size ranges from 1-4 kilobytes after minification and compression, representing the smallest overhead among the three approaches and proving negligible for even bandwidth-constrained environments. Memory usage remains minimal at approximately 5% above baseline, substantially lower than Redux Toolkit's 15% overhead. The learning curve proves gentlest among the alternatives with developers typically reaching productivity within 1-3 days due to the intuitive hooks-based API and minimal conceptual overhead (Dev. 2025).

Firebase integration with Zustand achieves elegant simplicity through store methods that initialize Firestore listeners, update Zustand state on snapshot events, and implement optimistic updates with rollback capabilities. The implementation pattern creates a Zustand store containing expense array state, listener initialization methods that establish `onSnapshot` subscriptions updating the store on Firestore changes, and action methods such as `addExpense` that optimistically update local state before writing to Firestore, with error handling rolling back optimistic updates if writes fail. This pattern provides responsive user interfaces with immediate feedback while maintaining consistency with Firestore's source of truth.

Zustand supports middleware including DevTools integration providing Redux DevTools compatibility, persistent middleware for localStorage synchronization, and immer middleware enabling mutable update syntax while maintaining immutability guarantees. The library's flexibility enables gradual adoption, with new stores added incrementally without requiring wholesale architecture changes. Testing proves straightforward through simple function mocking, avoiding the provider wrapper complexity of Context API or store configuration overhead of Redux. The primary limitation lies in Zustand's smaller ecosystem compared to Redux, though the library's maturity and growing community have produced comprehensive documentation and common pattern examples addressing typical use cases. (Dev. 2025)

9.4 Comparative analysis and recommendations

Comprehensive comparison across critical dimensions reveals clear differentiation among the three approaches. Bundle size ranges from zero for Context API (built into React) to 1-4 kilobytes for Zustand and 15 kilobytes for Redux Toolkit, though these differences prove negligible in absolute terms for most applications. Performance metrics favour Zustand with 35 millisecond update times compared to 65 milliseconds for Redux Toolkit and 75 milliseconds for Context API, though all three deliver acceptable performance for typical usage patterns. dev Development velocity heavily favours Zustand and Context API over Redux Toolkit due to reduced boilerplate and conceptual simplicity.

For expense-sharing applications in the small-to-medium scale category—those with 20-50 components serving teams of 2-5 developers—Zustand emerges as the optimal choice. (Dev. 2025) The library delivers excellent performance essential for responsive real-time updates, minimal bundle size supporting fast application load times, straightforward Firebase integration patterns, and sufficiently gentle learning curve enabling rapid development velocity crucial for student projects. The solution scales adequately to support growth from initial prototype through production deployment serving thousands of users without requiring architectural migration.

Context API remains appropriate only for applications with genuinely minimal state management needs, typically fewer than 20 components with infrequent state updates. Redux Toolkit justifies its complexity primarily for large enterprise applications exceeding 100

components with teams of 15+ developers requiring strict architectural patterns, extensive middleware integration, or sophisticated debugging capabilities for complex state interactions. (Dev. 2025) For Split-Sync, Zustand provides the ideal balance enabling rapid development while maintaining production-grade performance and maintainability characteristics essential for fintech applications handling financial data.

10.0 Offline Support Strategies and Implementation

Offline capability represents a critical feature for mobile expense-sharing applications, enabling users to record expenses immediately regardless of network availability. Modern Progressive Web App technologies combined with sophisticated data synchronization strategies enable web applications to deliver app-like experiences including offline functionality, background synchronization, and seamless conflict resolution. Educative

10.1 Progressive Web App architecture

Progressive Web Apps transform traditional web applications into installable applications with native-like capabilities through three core technologies: a web manifest defining app metadata, service workers enabling offline functionality, and HTTPS enforcing secure connections. React applications can be converted to PWAs using Create React App's PWA template (`npx create-react-app [name] --template cra-template-pwa`) providing pre-configured service worker files, (Dev. 2025) or through Vite with the vite-plugin-pwa package offering automatic service worker generation and update management.

The web app manifest, a JSON file defining application name, icons, display mode, theme colours, and start URL, enables users to install web applications to their device home screens. (Dev. 2025)When a user installs a PWA, the browser creates an app-like entry point launching the application in a standalone window without browser chrome, providing an immersive experience indistinguishable from native applications. Icon specifications require multiple sizes with 192x192 and 512x512 pixels representing minimum requirements for proper display across devices. Display modes including "standalone", "fullscreen", and "minimal-ui" control how the installed application appears, with standalone proving optimal for most applications by hiding browser UI while preserving system navigation.

HTTPS represents a mandatory requirement for service worker registration, as the powerful capabilities service workers provide—including intercepting all network requests and accessing cached resources—would represent severe security vulnerabilities over unencrypted connections. (Dev. 2025) Development environments typically exempt localhost from this requirement enabling local testing without certificate configuration, but production deployments must implement valid SSL certificates. Modern hosting platforms including Vercel, Firebase, and

Railway provide automatic HTTPS with certificate management, eliminating manual configuration burden.

10.2 Service worker caching strategies

Service workers intercept network requests enabling sophisticated caching strategies tailored to resource types and update frequencies. (Dev. 2025) Five primary strategies address different use case requirements. Cache-first strategy checks the cache before making network requests, returning cached responses immediately if available and falling back to network only on cache misses. (Dev. 2025) This approach delivers optimal performance for static assets including JavaScript bundles, CSS stylesheets, fonts, and images where freshness proves less critical than load speed.

Network-first strategy attempts network requests before falling back to cached responses, ensuring content freshness when connectivity exists while providing graceful degradation during offline periods. Nbellocam This pattern suits API responses and frequently updated content where serving stale data proves acceptable only when fresh data remains unavailable. Stale-while-revalidate strategy immediately returns cached responses while simultaneously fetching fresh content from the network and updating the cache, delivering both immediate response times and eventual consistency. This approach proves ideal for expense lists, user profiles, and other content where slightly stale data proves acceptable initially but should update automatically once fresh data arrives.

Cache-only and network-only strategies represent edge cases appropriate for specific scenarios. Cache-only serves exclusively from cache without network requests, suitable for application shells guaranteed to be pre-cached during service worker installation. Network-only bypasses cache entirely, appropriate for real-time data where caching would prove counterproductive or authentication endpoints where caching might create security vulnerabilities.

Google's Workbox library provides production-ready implementations of these strategies with minimal configuration. Workbox enables route-based strategy selection using request destination property, allowing different strategies for images, scripts, documents, and API calls. Expiration policies including maximum cache entries and maximum cache age prevent unbounded cache growth consuming device storage. Background sync capabilities enable deferring failed requests until connectivity restores, with automatic retry logic and progress notifications maintaining user awareness.

10.3 Data synchronization and conflict resolution

Offline-first architecture requires careful consideration of data synchronization patterns and conflict resolution strategies. The fundamental challenge arises when multiple users or devices

modify the same data while disconnected, creating conflicting versions that must be reconciled when connectivity restores. Several conflict resolution strategies address different use cases with varying complexity and guarantees.

Last-write-wins (LWW) represents the simplest approach, assigning timestamps or unique identifiers to each modification with the highest value prevailing during conflict resolution. This strategy proves acceptable for user profiles, application preferences, and other data where occasional overwrites prove tolerable. However, LWW proves inappropriate for financial data where data loss represents unacceptable risk. Expense sharing specifically requires more sophisticated approaches preventing balance calculation errors.

Conflict-Free Replicated Data Types (CRDTs) represent data structures mathematically designed to support conflict-free merging regardless of operation order. CRDTs prove particularly valuable for collaborative applications where multiple users simultaneously edit shared data. G-Counter (grow-only counter) CRDTs handle total expense tracking by maintaining per-replica counts with merge operations taking maximum values per replica. OR-Set (observed-remove set) CRDTs manage participant lists by tracking additions with unique identifiers and removals referencing those identifiers, enabling addition and removal operations to commute without conflicts.

For financial data including expense amounts and balance calculations, additive resolution proves most appropriate. Rather than choosing between conflicting values, additive resolution sums differences representing both modifications. If two devices simultaneously record different expense amounts for the same user, both expenses should be recorded as separate transactions rather than one overwriting the other. This approach requires storing operations (deltas) rather than final values, enabling reconstruction of complete transaction history and accurate balance calculation regardless of synchronization order.

Manual conflict resolution through user intervention represents the final strategy for high-value transactions or complex business logic where automated resolution risks incorrect outcomes. When conflicts arise, applications present both conflicting versions enabling users to select the correct value, merge information manually, or discard one version. This approach provides maximum correctness at the cost of user burden and interaction delays. For Split-Sync, manual resolution should activate only for conflicts involving amounts exceeding predefined thresholds or transactions marked as critical by users.

10.4 Firebase offline capabilities and limitations

Firebase provides integrated offline support through Firestore offline persistence, automatically caching documents accessed from the backend and synchronizing local modifications when connectivity restores. The default 100 megabyte cache proves sufficient for most applications,

with configuration options enabling unlimited cache for data-intensive applications. Multi-tab support requires explicit configuration through `persistentMultipleTabManager()` enabling state synchronization across browser tabs, essential for users who might have expense-sharing applications open in multiple windows simultaneously.

Firestore's offline mode enables reading and writing data with the same API regardless of connectivity status. Applications detect whether data originates from cache or server through metadata inspection using `metadata.fromCache` property. Write operations queue automatically during offline periods with Firestore applying operations to the backend once connectivity restores. This automatic synchronization dramatically simplifies offline implementation compared to manual queue management and retry logic.

Critical limitations require architectural consideration. Firestore transactions do not persist across application restarts; if a transaction begins while offline and the application closes before connectivity restores, the transaction must be re-executed. This limitation necessitates careful transaction design ensuring idempotency to prevent duplicate execution. Additionally, Firestore's conflict resolution employs last-write-wins semantics, potentially problematic for financial data as discussed previously. Applications requiring different conflict resolution strategies must implement custom logic detecting conflicts through document versioning and resolving appropriately based on business logic.

Storage quotas present practical limitations for offline-capable applications. Chrome and Chromium browsers typically allocate 60-80% of available disk space to browser storage with approximately 20% of that allocation available per origin, translating to roughly 6.6 gigabytes per origin on devices with 100 gigabytes free space. Firefox proves more restrictive with group limits of 10% of disk or 10 gigabytes (whichever proves smaller) and per-origin limits of 20% of the group limit. Safari implements the most aggressive policies with iOS limiting IndexedDB to 500 megabytes to 1 gigabyte and proactively evicting data after seven days without user interaction. Applications must monitor storage usage through the Storage Management API's `navigator.storage.estimate()` method and implement data cleanup strategies preventing quota exhaustion.

10.5 Security implications of offline storage

Storing financial data locally introduces security considerations requiring careful mitigation. Local storage including IndexedDB, Cache Storage, and localStorage remains unencrypted by default, accessible to any JavaScript code executing within the application's origin. Cross-Site Scripting (XSS) vulnerabilities enabling injection of malicious JavaScript could expose all locally stored data. Applications must never store highly sensitive data including passwords, complete credit card numbers, social security numbers, or unencrypted authentication tokens in client-side storage.

Offline-capable expense applications should store only data necessary for core functionality: expense descriptions, amounts, dates, and participant information. Authentication tokens should employ HTTP-only cookies preventing JavaScript access, or store in memory with short lifespans requiring re-authentication after browser closure. When sensitive data storage proves unavoidable, applications should employ Web Crypto API for encryption before storage, though key management introduces additional complexity as encryption keys themselves require secure storage.

Logout operations must comprehensively clear all locally cached data, deleting all caches using `caches.delete()` for each cache name, destroying local databases through database-specific APIs, and clearing session storage. This thorough cleanup prevents data persistence after user logout and protects users on shared devices. Periodic re-authentication for sensitive operations including large expense creation, participant addition, or balance settlement provides additional security layers beyond initial authentication.

10.6 User experience patterns for offline states

Effective offline support requires clear communication of connectivity status and synchronization state to users. Visual indicators should employ multiple sensory channels to accommodate accessibility requirements; colour-coded dots (green for online, gray for offline, yellow for synchronizing) should accompany text labels and icons avoiding reliance on colour perception alone. Connection status changes should trigger toast notifications providing transient feedback without disrupting workflow, with messages such as "You are offline. Changes will sync when reconnected" setting appropriate expectations.

Sync status indicators should distinguish between saved locally, currently synchronizing, successfully synced, and sync failed states through icons and micro-copy. Expense items might display small icons indicating sync status with checkmarks representing successful synchronization, cloud icons with arrows representing active synchronization, and warning icons with retry buttons representing failed synchronization. This granular feedback enables users to understand exactly which changes have propagated to the server and which remain local, building confidence in data persistence.

Offline capability indicators help users understand which features remain available without connectivity. Download icons adjacent to expenses indicate offline availability, while grayed-out controls communicate features requiring network connectivity. Empty states should provide context-appropriate messaging; offline empty states might explain "Your expenses will appear here once you're back online" while error states provide actionable next steps such as retry buttons or instructions for manual data refresh.

11.0 Essential Development Tools and Workflow

Professional development workflows incorporate tools and practices ensuring code quality, facilitating collaboration, preventing security vulnerabilities, and maintaining comprehensive documentation. Student developers building fintech applications benefit substantially from adopting industry-standard tooling, both for the immediate project quality improvements and for developing marketable skills valuable in professional contexts.

11.1 Version control with Git and branching strategies

Git represents the universal standard for version control with GitHub serving as the dominant hosting platform. Branching strategy selection significantly impacts team collaboration efficiency and merge conflict frequency. Trunk-based development has emerged as the preferred approach for small-to-medium teams, employed by industry leaders including Google with 35,000+ developers. The model employs a single main branch where all developers commit at minimum daily, with feature branches remaining short-lived at fewer than two days duration or developers committing directly to trunk with feature flags hiding incomplete functionality.

Trunk-based development's advantages include dramatically reduced merge conflicts through frequent integration, faster feedback loops enabling rapid identification of integration issues, minimal branch management overhead freeing developers to focus on features rather than Git operations, and natural support for continuous integration and deployment. The model proves particularly well-suited for student teams of 2-10 developers working on time-constrained projects where simplicity and velocity outweigh the structured release management benefits of more complex branching models.

GitFlow presents an alternative multi-branch model with long-lived develop and main branches supplemented by short-lived feature, release, and hotfix branches. This structured approach suits larger teams with scheduled release cycles and formal testing phases, but introduces substantial overhead for smaller teams through increased merge complexity and branch management burden. For Split-Sync, trunk-based development combined with feature flags for incomplete features provides optimal simplicity while maintaining code quality through pre-commit hooks and automated testing.

11.2 Testing strategy and framework selection

Comprehensive testing forms the foundation of fintech application quality, with financial institutions typically requiring 80-90% overall code coverage and 90-100% coverage for critical financial logic including balance calculations, transaction processing, and data integrity operations. The testing pyramid architecture allocates roughly 60% effort to unit tests verifying

individual functions and components, 30% to integration tests validating interactions between modules, and 10% to end-to-end tests confirming complete user workflows.

Jest serves as the de facto standard for JavaScript unit and integration testing, providing zero-configuration integration with Create React App, fast parallel test execution, built-in code coverage reporting, sophisticated mocking capabilities, and snapshot testing for component output verification. React Testing Library complements Jest by testing components from user perspectives, emphasizing behaviour over implementation details and promoting accessible design through queries preferring semantic HTML attributes over implementation-specific test IDs. The library's `screen.getByRole()` and `screen.getByLabelText()` queries encourage proper ARIA attribute usage, improving application accessibility as a testing side effect.

Cypress provides modern end-to-end testing through a visual test runner executing tests in real browsers with automatic waiting for asynchronous operations, time-travel debugging enabling inspection of application state at each test step, and comprehensive network stubbing for reliable test execution. Cypress excels at validating complete user journeys including authentication flows, expense creation with multiple participants, balance calculation verification, and payment settlement workflows. The visual feedback during test execution proves invaluable for diagnosing failures and understanding user experience implications of interface changes.

Test coverage configuration should enforce higher thresholds for financial logic through Jest's coverage threshold settings. Global thresholds might require 80% coverage for branches, functions, and lines, while directory-specific overrides enforce 95% coverage for utility functions handling financial calculations. Continuous integration pipelines should fail builds failing to meet these thresholds, preventing gradual erosion of test coverage as projects evolve.

11.3 API documentation with Swagger and OpenAPI

Comprehensive API documentation proves essential for fintech applications, facilitating frontend-backend coordination, enabling third-party integrations, supporting audit and compliance requirements, and serving as reference during debugging and troubleshooting. Swagger and OpenAPI have emerged as industry standards for REST API documentation, with OpenAPI 3.0 representing the current specification and Swagger providing tooling for generating interactive documentation from OpenAPI specifications.

Node.js and Express applications integrate Swagger through the swagger-jsdoc and swagger-ui-express packages, enabling in-code documentation through JSDoc comments decorating route handlers. Developers document endpoints by specifying HTTP methods, path parameters, request body schemas, response schemas with status codes, and authentication requirements through structured comments that swagger-jsdoc parses into OpenAPI

specifications. The swagger-ui-express package renders these specifications as interactive HTML documentation accessible at a designated route such as `/api-docs`, enabling developers to test endpoints directly from their browsers by providing parameters and authentication credentials.

Benefits extend beyond human-readable documentation. OpenAPI specifications enable automatic client SDK generation for multiple programming languages, facilitating mobile application development or third-party integrations. Schema validation libraries can programmatically verify request and response payloads conform to documented specifications, catching discrepancies between documentation and implementation. API testing tools including Postman import OpenAPI specifications to generate automated test suites. For student projects, Swagger documentation provides professional polish demonstrating attention to detail and architectural consideration.

11.4 Monitoring, error tracking, and observability

Production applications require comprehensive monitoring and error tracking enabling rapid identification and diagnosis of issues. Sentry has emerged as the industry standard error tracking platform, offering generous free tiers supporting 5,000 errors monthly alongside 10,000 performance monitoring events. The platform provides detailed error reports including complete stack traces, user context, breadcrumbs documenting actions leading to errors, and release tracking enabling correlation of errors with specific deployments.

Integration requires minimal code with Sentry SDKs installed for both React frontend and Node.js backend. Configuration should specify environment (development, staging, production) enabling filtering and separate error quotas per environment. Performance monitoring through transaction tracing identifies slow database queries, API calls, and rendering operations enabling targeted optimization. Release tracking through Git commit SHAs in Sentry configuration enables automatic association of errors with specific code versions, dramatically accelerating issue identification and resolution.

Best practices include attaching hashed user identifiers (not personally identifiable information) to errors enabling user-specific issue diagnosis while maintaining privacy compliance, tagging errors by feature module facilitating identification of problematic components, and implementing `beforeSend` hooks scrubbing sensitive data from error reports before transmission to Sentry servers. Breadcrumb customization should capture application-specific context such as expense amounts being processed or participant counts, providing debugging context beyond generic browser events.

11.5 Analytics implementation with privacy consideration

User behaviour analytics inform product development decisions, identify usability issues, and measure feature adoption. However, fintech applications must balance analytics value against privacy obligations, particularly GDPR compliance for European users. Google Analytics 4 faces substantial GDPR compliance challenges due to data transfers to US servers, with Austrian, French, Italian, and Danish regulators declaring Google Analytics illegal without explicit user consent and additional safeguards.

Mixpanel represents a privacy-friendly alternative offering user-centric tracking focused on individual user journeys rather than aggregate traffic statistics. The platform provides unlimited funnel analysis enabling detailed conversion rate optimization, real-time data eliminating the 24-hour delays characteristic of Google Analytics, and EU data residency options ensuring data remains within European regulatory jurisdiction. The generous free tier supporting 20 million monthly events proves more than sufficient for student projects. GDPR-compliant configuration requires specifying the EU API endpoint through the `api_host` parameter, disabling IP address collection through `ip: false` configuration, and implementing property blacklists preventing accidental collection of email addresses or other personally identifiable information.

Cookie consent implementation through libraries such as react-cookie-consent ensures regulatory compliance by preventing analytics initialization until users explicitly accept. Consent banners should employ clear language explaining data collection purposes, provide easily accessible decline options, and respect user preferences across sessions through localStorage persistence. Privacy policies must accurately describe analytics implementations including data retention periods, third-party data sharing (if any), and procedures for users requesting data deletion. For Split-Sync, implementing Mixpanel with EU data residency and comprehensive cookie consent provides powerful analytics while maintaining GDPR compliance essential for fintech applications.

11.6 Development workflow automation

Automated code quality enforcement prevents technical debt accumulation and maintains consistent code style across development teams. ESLint provides configurable JavaScript and TypeScript linting enforcing code quality rules, identifying potential bugs, and standardizing code style. Prettier complements ESLint through opinionated code formatting, automatically handling indentation, line length, quote style, and other syntactic concerns. Husky integrates with Git hooks enabling pre-commit automation that prevents commits failing to meet quality standards.

The complete workflow installs ESLint with appropriate presets for React and TypeScript, Prettier with ESLint integration preventing rule conflicts, Husky for Git hook management, and lint-staged for efficient pre-commit checks on only modified files. Configuration files define linting rules appropriate for the project, Prettier formatting preferences, and lint-staged

specifications determining which tools process which file types. The pre-commit hook configured through Husky invokes lint-staged, which runs ESLint with automatic fixing and Prettier formatting on staged files, preventing commit completion if errors remain.

Package.json scripts provide convenient commands for manual invocation: `lint` executes ESLint across the entire codebase identifying issues, `lint:fix` automatically resolves fixable issues, and `format` applies Prettier formatting to all files. These commands integrate into continuous integration pipelines ensuring that merged code maintains quality standards even if developers bypass pre-commit hooks. Visual Studio Code extensions for ESLint and Prettier provide real-time feedback during development with inline error highlighting and format-on-save capabilities configured through workspace settings.

Essential VS Code extensions for React development include ESLint for inline linting, Prettier for automatic formatting, ES7+ React snippets for rapid component scaffolding, GitLens for enhanced Git integration, npm Intellisense for import autocompletion, Jest for test integration, and Thunder Client for API testing. These extensions dramatically improve development velocity and code quality through immediate feedback and automated assistance.

11.7 Dependency management and security

NPM dependency management requires careful security consideration, particularly for fintech applications where vulnerable dependencies might expose user data or financial information. Best practices begin with enabling two-factor authentication on NPM accounts preventing unauthorized package publication under developer credentials. Regular security audits through `npm audit` identify known vulnerabilities in dependency trees with `npm audit fix` automatically updating to patched versions where available. The Snyk platform provides more comprehensive vulnerability scanning with a free tier supporting 200 monthly tests, continuous monitoring for newly disclosed vulnerabilities, and automated pull requests fixing identified issues.

Continuous integration pipelines must use `npm ci` rather than `npm install` to ensure reproducible builds from lockfile specifications without updating package versions. For fintech applications, pinning exact versions in package.json (using `"express": "4.18.2"` rather than `"express": "^4.18.2"`) prevents unexpected updates introducing vulnerabilities or breaking changes. Regular dependency updates should occur through deliberate processes including reviewing changelogs, testing thoroughly in staging environments, and monitoring error rates after production deployment.

Dependabot or Renovate Bot automate dependency update management through automated pull requests proposing version updates with release notes and changelog links. These tools enable

proactive security patch application while maintaining visibility and control over dependency changes. Security configuration should include .npmignore files preventing accidental publication of environment variables or secrets, npm publishConfig restricting package registry to official NPM, and engine specifications ensuring consistent Node.js and NPM versions across development environments.

For Split-Sync specifically, this comprehensive tooling strategy remains entirely free, with all recommended tools offering generous free tiers sufficient for student project requirements. The investment in tooling setup pays immediate dividends through improved code quality, faster debugging, reduced security vulnerabilities, and valuable professional skill development. The resulting application demonstrates production-grade quality appropriate for portfolio presentation and professional discussion, distinguishing student work through professionalism and attention to engineering excellence.

12. Conclusion

The SplitSync project represents an innovative and technically grounded solution to a persistent problem in everyday group finance management. By combining automation, intelligent data processing, and seamless payment integration, it addresses the limitations of existing systems such as Splitwise, Tikkie, and Settle Up. Through its modular architecture, real-time capabilities, and cross-platform design, SplitSync embodies the principles of modern software engineering — scalability, usability, and maintainability.

This research has demonstrated not only the feasibility of developing such a system but also the importance of user-centered design and technological innovation in financial applications. The implementation of SplitSync will not only simplify expense sharing but also serve as a testament to how emerging technologies like OCR and real-time APIs can reshape digital collaboration in finance.

As the financial technology landscape continues to evolve, SplitSync stands as a prototype of what the next generation of shared expense systems can achieve — a platform that is smarter, faster, and more intuitive, delivering value through both technological excellence and practical utility.

Appendix

Fig. 1 Tikkie Interfaces
Fig. 2 ScreenShot of Settle Up
Fig. 3 Revolut Interfaces
Fig. 4 Wise UI pictures
Fig. 5 Splitwise

References

ABN AMRO. (2024). *Tikkie payments reach record 7.4 billion euros in 2024*. ABN AMRO Bank. https://www.abnamro.com/en/news/tikkie-payments-reach-record-7-4-billion-euros-in-2024?utm_source=chatgpt.com [accessed 9 October 2025]

AbnAsia.org. (2024, December 18). Architecture of a Neobank - Revolut. *ABN Software*. https://news.abnasia.org/blog/posts/en-architecture-of-a-neobank-revolut-3689 [accessed 13 October 2025]

Abramowski, N. (2025, August 5). What are Tech Stacks? Choosing the Right One. CareerFoundry. Retrieved 20 October. 2025, from https://wikipedia.comcareerfoundry.com/en/blog/web-development/what-is-a-tech-stack/#:~:text=A%20tech%20stack%20is%20short%20for%20technology%20stack.,from%20frontend%20to%20backend%2C%20to%20build%20an%20application.

AIMultiple. (2025). OCR Benchmark: Text Extraction / Capture Accuracy. Wikipedia. [online] Available at: <https://research.aimultiple.com/ocr-accuracy/> [Accessed 20 October. 2025].

Andersen, G. (2025). How to Overcome Common Security Issues in Fintech App Development - Essential Strategies and Best Practices. *MoldStud - Custom Software Development Company*. https://moldstud.com/articles/p-how-to-overcome-common-security-issues-in-fintech-app-development-essential-strategies-and-best-practices [accessed 19 October 2025]

Atlassian (2024) *Trunk-based Development*. Available at: https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development (Accessed: 19 October 2025).

AWS. (2025). *Full Stack Development Service - Free Amazon Amplify - AWS*. Amazon Web Services, Inc. https://aws.amazon.com/pm/amplify/

Ayres, E. (2019). Best Completely FREE Cloud Hosting Services [2026]. *WebsitePlanet*. https://www.websiteplanet.com/blog/best-free-cloud-hosting-services/ (Accessed 12 October 2025).

BIO. (2025). *Contactless payments continue to rise, ATM transactions fall*. Bank of Ireland Group Website. https://www.bankofireland.com/about-bank-of-ireland/press-releases/2025/contactless-payments-continue-to-rise-atm-transactions-fall/ [accessed 6 October 2025]

Bittner, J. (2019, July 9). *Splitwise, redesigned*. The Splitwise Blog. https://blog.splitwise.com/2019/07/09/splitwise-redesigned/ [accessed 10 October 2025]

Chakraborty, S. (2024) 'Zustand vs. Redux Toolkit - A Comprehensive Comparison in State Management', *Subhojit.me*. Available at: https://www.subhojit.me/blog/zustand-vs-redux-toolkit-a-comprehensive-comparison-in-state-management/ (Accessed: 19 October 2025).

Das, P., 2023, March. *Automation and Transparency in FinTech*. ResearchGate

De Best, R. (2024). *Netherlands: Tikkie users 2016-202*. Statista. https://www.statista.com/statistics/917381/number-of-users-of-tikkie-in-the-netherlands/?utm_source=chatgpt.com

Dev. 2025, State Management in 2025: When to Use Context, Redux, Zustand, or Jotai - DEV Community. Dev. [online] Available at: <https://dev.to/hijazi313/state-management-in-2025-when-to-use-context-redux-zustand-or-jotai-2d2k> [Accessed 20 October. 2025].

Drozdovica, J. (2024).Types of Payment Gateway: Use Cases and Benefits. https://noda.live/articles/types-of-payment-gateway. [accessed 19 October 2025]

European Central Bank. (2023). Study on digital wallet features. *Europa*.

Firebase (2024) *Access data offline | Firestore*. Available at: https://firebase.google.com/docs/firestore/manage-data/enable-offline (Accessed: 19 October 2025).

Firebase (2024) *Firebase Authentication Pricing*. Available at: https://firebase.google.com/pricing (Accessed: 19 October 2025).

Frontegg (2024) *OAuth 2.0 vs JWT: Understanding the Key Differences*. Available at: https://frontegg.com/blog/oauth-vs-jwt (Accessed: 19 October 2025).

GeeksforGeeks. 2025, August 5. What are Tech Stacks? Choosing the Right One. [online] Available at: <https://wikipedia.comgeeksforgeeks.org/blogs/what-are-tech-stacks-choosing-the-right-one/> [Accessed 20 October. 2025].

Google (2024) *Cloud Vision API Documentation*. Available at: https://cloud.google.com/vision/docs (Accessed: 19 October 2025).

Google Design (2024) *UX Design for Offline: Build Anywhere*. Available at: https://design.google/library/offline-design (Accessed: 19 October 2025).

Gottumukkala., M. (2025). Payment Gateway Types: A Comprehensive Breakdown for E-commerce Businesses. Aasaan. https://aasaan.app/blog/types-of-payment-gateways/

Grigaliūnas, Š., Malinauskaitė, I.,雅各布斯, J. and Cenys, A. (2025) 'Security Challenges in Financial Technology', in *Information and Software Technologies: 30th International Conference, ICIST 2024*. Springer, pp. 386-400.

HamzaKhan07. (n.d.). *GitHub - HamzaKhan07/SettleUp*. GitHub. Retrieved October 16, 2025, from https://github.com/HamzaKhan07/SettleUp?tab=readme-ov-file

Hoang, C. (2025). Best Completely FREE Cloud Hosting Services [2025]. WebsitePlanet. Retrieved 20 October. 2025, from https://wikipedia.comwebsiteplanet.com/blog/best-free-cloud-hosting-services/.

Hugo. (2025). Tikkie DE [Video]. In *Dribbble*. https://dribbble.com/shots/17463411-Tikkie-DE

Huth, S. (2024). *The story of Wise and Bitrise*. Bitrise. https://bitrise.io/customer-stories/transferwise

Iyer, S . (2025). How to maintain security compliance at a Fintech: A complete guide. Wikipewikpedia. Retrieved 20 October. 2025, from https://wikipewikpedia. (2025). Payment Card Industry Data Security Standard. Wikipedia. Retrieved 20 October. 2025, from wikipedia.comgeeksforgeeks.org/blogs/what-are-tech-stacks-choosing-the-right-one/.dia.com.

James, M. (2025). *Wise Tech Stack (2025 update)*. Wise Engineering. https://medium.com/wise-engineering/wise-tech-stack-2025-update-d0e63fe718c7 [accessed 19 October 2025]

Kairov, Y. (2024). *(30) Post*. LinkedIn. https://www.linkedin.com/posts/yunus-kairov_software-engineer-java-fincrime-revolut-activity-7204814587932610561-crbR/?utm_source=chatgpt.com [accessed 13 October 2025]

Koopal, H. S. (2024). *Tikkie payments reach record 7.4 billion euros in 2024*. ABN AMRO Bank.[online] https://www.abnamro.com/en/news/tikkie-payments-reach-record-7-4-billion-euros-in-2024 [accessed 08 October 2025].

Kumthekar, T. (2018, May 25). *Splitwise: a UX case study*. UX Collective. [online] https://uxdesign.cc/splitwise-a-ux-case-study-dc2581971226 [accessed 9 October 2025]

Levi, G. (2025). 7 Types of Payment Gateways to Elevate Your Business. Sensepass. https://sensepass.com/7-types-of-payment-gateways/

Lvivity. (2024). How to integrate a payment gateway into your mobile phone. Lvivity. https://lvivity.com/payment-gateway-into-mobile-app

Mahesh, H. (2025). *Test Automation for FinTech Applications: Best Practices*. testRigor AI-Based Automated Testing Tool. https://testrigor.com/blog/test-automation-for-fintech-applications/

Mundra, P. (2025). State Management: Comparing Redux Toolkit, Zustand, and React Context. *Prakash Software Solutions*. https://prakashinfotech.com/state-management-comparing-redux-toolkit-zustand-and-react-context

Nesbit, J. (2025). *The 6 Best Apps for Splitting Bills With Friends*. HerMoney. [online] https://hermoney.com/connect/friends/best-apps-for-splitting-bills-with-friends/ [accessed 6 October 2025]

Oleg. (2025). How to choose the best payment gateway for mobile app and integrate it. https://touchlane.com/how-to-choose-the-best-payment-gateway-for-mobile-app-and-integrate-it/

OWASP (2025) OWASP Top Ten Web Application Security Risks. *OWASP Foundation*. [online]. Available from: https://owasp.org/www-project-top-ten/ [Accessed January 23, 2026].

React. (2025). Create React App is deprecated. Wikipedia. [online] Available at: <https://wikipedia.comCreate React App is deprecated> [Accessed 20 October. 2025].

Revolut. (2021). *The fundamentals of web development at Revolut*. Revolut Tech. https://medium.com/revolut/the-fundamentals-of-web-development-at-revolut-bb5e2888e84f

Reddit. (2020). *Incoming Features : r/Revolut*. Reddit. https://www.reddit.com/r/Revolut/comments/ha8pnc/incoming_features/

Redux Team (2024) *Redux Toolkit*. Redux Toolkit Documentation. Available at: https://redux-toolkit.js.org/ (Accessed: 19 October 2025).

Robinson, L. (2025) How can I use GitHub Actions with Vercel? *Vercel Knowledge Base*. [online]. Available from: https://vercel.com/kb/guide/how-can-i-use-github-actions-with-vercel [Accessed March 23, 2026].

Scheffler, P. (2025). *Fintech App Security: Building a Secure Fintech App*. Neontri. https://neontri.com/blog/fintech-app-security/ (Accessed: 20 October, 2025).

SettleUp. (2025). *Settle Up*. Settle Up. https://settleup.io/tips.html

Sharma, V. (2007) Payment Gateway integration in e-commerce: A complete step-by-step Guide. SparxIT. https://www.sparxitsolutions.com/blog/payment-gateway-integration/

Stage.nl. (2025). *Tikkie Backend Developer in Amsterdam - Stage.nl*. Stage. https://www.stage.nl/stages/2293075-tikkie-backend-developer-in-amsterdam?utm_source=chatgpt.com

Splitwise. (2025). *Splitwise Pro*. [online] Splitwise. https://www.splitwise.com/pro [accessed 08 October 2025].

StripeDocs. (2025). Integration security guide. Wikipedia. Retrieved 20 October. 2025, from https://wikipedia.comdocs.stripe.com/security/guide.

SuperTokens. (2025, September 2). Explore Firebase's free vs Blaze tiers, real-world cost examples, hidden charges, and compare with Auth0/Okta for transparent pricing clarity. *SuperTokens*. https://supertokens.com/blog/firebase-pricing

Team, W. C. (2025). Fintech Encryption Standards. *WallStreetMojo*. https://www.wallstreetmojo.com/fintech-encryption-standards/

Tan, L., 2024, February. *The Future of Financial Collaboration*. KAI Research.

Team, D. (2025). Data Encryption in Transit and at Rest. *OceanoBe Technology*. https://oceanobe.com/news/data-encryption-in-transit-and-at-rest/1553

UK Finance. (2025). *Over half of UK adults now use mobile wallets*. UK Finance. https://www.ukfinance.org.uk/news-and-insight/press-release/over-half-uk-adults-now-use-mobile-wallets

Vávra, D. (2017). *What's new in the new Settle Up for Android?* Step Up Labs. https://medium.com/step-up-labs/whats-new-in-the-new-settle-up-for-android-ea53bfabd96f

Victor, Uma. (2025). Fintech Security: Best Practices for Fintech Apps. *DEV Community*. https://dev.to/flutterwaveeng/fintech-security-best-practices-for-fintech-apps-4k2i [Accessed 19 October 2025].

Watson, H. (2025). *Essential Fintech App Security Solutions Every Developer Must Know*. Medium. Available at https://medium.com/@.hazelwatson/essential-fintech-app-security-solutions-every-developer-must-know-4b0e7f42dbf7 (Assessed: 15 October 2025).

Wikipedia. (2025). Payment Card Industry Data Security Standard. Wikipedia. Retrieved 20 October. 2025, from https://wikipedia.comgeeksforgeeks.org/blogs/what-are-tech-stacks-choosing-the-right-one/.